

Investigación e Implementación de la Virtualización de una Estación Inteligente de Monitoreo Ambiental basada en STM32

Documento de Investigación y Desarrollo

10 de mayo de 2026

Resumen

La nueva revolución de las tecnologías de la información, representada por el Internet de las Cosas (IoT) y la tecnología embebida, está reconfigurando el modelo de funcionamiento social. El monitoreo ambiental inteligente se ha convertido en un componente clave de la construcción de ciudades inteligentes. Sin embargo, en el proceso de enseñanza e investigación y desarrollo de sistemas embebidos, los desarrolladores enfrentan desafíos como altos costos de hardware, procesos de depuración complejos, alta barrera de comprensión de sensores y dificultades en la visualización de datos. El modelo tradicional de desarrollo embebido requiere que los estudiantes adquieran una gran cantidad de dispositivos de hardware y realicen depuraciones repetidas en circuitos reales, lo que no solo aumenta el costo de aprendizaje, sino que también limita la flexibilidad y escalabilidad de los escenarios de enseñanza.

Para abordar los problemas mencionados, este artículo propone e implementa un sistema de virtualización de una estación inteligente de monitoreo ambiental basada en STM32. Las contribuciones principales de esta investigación incluyen tres aspectos: en primer lugar, se diseñó e implementó una plataforma de hardware virtual STM32F103C8T6 de alta fidelidad, que simula completamente el núcleo del MCU, el sistema de memoria, GPIO, ADC, I2C, UART y temporizadores, proporcionando un entorno de software donde los programas embebidos pueden ejecutarse sin necesidad de hardware real; en segundo lugar, basado en la plataforma de hardware virtual, se desarrollaron controladores virtuales para seis tipos comunes de sensores ambientales, incluyendo el sensor de temperatura y humedad DHT11, el sensor de calidad del aire MQ-135, el sensor de luz BH1750, el sensor de presión BMP280 y el sensor de ruido, estableciendo un marco de modelado de sensores unificado y un modelo de ruido para garantizar la autenticidad y confiabilidad de los datos simulados; finalmente, se construyó un sistema de visualización Web basado en Python Flask, que implementa funciones de adquisición de datos de sensores en tiempo real, almacenamiento, alertas y visualización de gráficos, interactuando con el frontend a través de API RESTful y soportando el control remoto de parámetros ambientales.

Los resultados experimentales muestran que la eficiencia de ejecución de instrucciones de la plataforma virtual STM32 tiene una alta consistencia con el hardware real, la precisión de simulación de sensores alcanza o se aproxima a los valores especificados en las hojas de datos de los sensores reales, y el tiempo de respuesta de la API del sistema Web es menor a 100 milisegundos, satisfaciendo los requisitos de monitoreo en tiempo real. Esta investigación proporciona una plataforma experimental de bajo umbral y alta flexibilidad para la enseñanza de sistemas embebidos, así como un entorno eficiente de desarrollo de software para la verificación de prototipos de aplicaciones IoT, generando valor tanto a nivel teórico como de ingeniería.

Palabras clave: STM32; virtualización; monitoreo ambiental; simulación de sensores;

Internet de las Cosas; enseñanza de sistemas embebidos

Índice

Resumen	1
1 Introducción	6
1.1 Antecedentes y Significado de la Investigación	6
1.2 Estado Actual de la Investigación Nacional e Internacional	7
1.2.1 Investigación en Plataformas de Simulación Embebida	7
1.2.2 Investigación en Sistemas de Monitoreo Ambiental	8
1.3 Trabajo Principal de la Tesis	9
1.4 Estructura de la Tesis	10
2 Diseño General del Sistema	11
2.1 Análisis de Requisitos Funcionales	11
2.1.1 Requisitos Funcionales	11
2.1.2 Requisitos No Funcionales	12
2.2 Diseño de la Arquitectura General	12
2.3 Selección de Tecnologías	14
3 Diseño de la Plataforma de Hardware Virtual STM32	15
3.1 Diseño del Núcleo Virtual MCU	15
3.1.1 Resumen de Parámetros del STM32F103C8T6	15
3.1.2 Simulación del Temporizador SysTick	16
3.2 Diseño del Sistema de Memoria	17
3.2.1 Modelo de Mapeo de Memoria	17
3.2.2 Implementación del Modo Little-Endian	18
3.2.3 Mapeo de Registros	18
3.3 Simulación de Periféricos	19
3.3.1 GPIO Entrada/Salida de Propósito General	19
3.3.2 ADC Convertidor Analógico-Digital	19
3.3.3 Controlador de Bus I2C	20
3.3.4 Comunicación UART Puerto Serie	21
3.3.5 TIM Temporizador General	23
4 Controladores de Sensores y Adquisición de Datos	24
4.1 Principios de Virtualización de Sensores	25
4.1.1 Marco de Modelado General	25
4.1.2 Interfaz del Controlador de Sensores	26
4.2 Sensor de Temperatura y Humedad DHT11	26

4.2.1	Descripción General del Sensor	26
4.2.2	Simulación del Protocolo de Bus Único	27
4.3	Sensor de Calidad del Aire MQ-135	28
4.3.1	Principio del Sensor	28
4.3.2	Inversión Voltaje-Concentración	29
4.3.3	Implementación del Controlador Virtual	29
4.4	Sensor de Luz BH1750	30
4.4.1	Descripción General del Sensor	30
4.4.2	Fórmula de Cálculo de Iluminancia	31
4.5	Sensor de Presión BMP280	31
4.5.1	Descripción General del Sensor	31
4.5.2	Verificación del ID del Chip	31
4.5.3	ADC de 24 bits y Coeficientes de Calibración	32
4.6	Sensor de Ruido	32
4.6.1	Principio del Sensor	32
4.6.2	Mapeo Lineal de Voltaje	33
4.7	Diseño del Ciclo de Adquisición de Datos	34
5	Diseño e Implementación del Sistema de Software	36
5.1	Módulo de Almacenamiento de Datos	36
5.1.1	Diseño de la Estructura de Tablas de la Base de Datos	36
5.1.2	Interfaz sqlite3 de Python	38
5.1.3	Operaciones CRUD	38
5.1.4	Consultas Estadísticas	39
5.2	Diseño del Sistema de Alertas	39
5.2.1	Configuración de Umbrales de Alerta	40
5.2.2	Lógica de Verificación de Alertas	41
5.2.3	Persistencia de Registros de Alertas	42
5.3	Sistema de Visualización Web	43
5.3.1	Diseño de Rutas Flask	43
5.3.2	Formato de Respuesta JSON RESTful	44
5.3.3	Configuración de Gráfico de Líneas Dual de Chart.js	45
5.3.4	Mecanismo de Sondeo de 1 Segundo	46
5.4	Diseño del Panel de Control	47
5.4.1	Diseño del Mecanismo IPC	47
5.4.2	Análisis del Comando set	48
5.4.3	Restricciones del Rango de Parámetros	49

6	Pruebas y Verificación del Sistema	50
6.1	Entorno de Pruebas	50
6.2	Pruebas Funcionales	50
6.3	Pruebas de Rendimiento	51
6.3.1	Tiempo de Respuesta de la API	52
6.3.2	Estabilidad del Período de Adquisición	52
6.3.3	Uso de Memoria	52
6.4	Verificación de Precisión de Sensores	53
6.4.1	Análisis de Comparación de Precisión	53
6.4.2	Pruebas de Estabilidad a Largo Plazo	54
6.5	Verificación de la Función de Alertas	54
6.5.1	Escenario de Alerta de Alta Temperatura	54
6.5.2	Escenario de Alerta de Baja Presión	55
6.5.3	Escenario de Alertas Concurrentes Múltiples	55
6.6	Visualización de la Interfaz Web	56
7	Resumen y Perspectivas	57
7.1	Resumen del Trabajo de la Tesis	57
7.2	Perspectivas de Trabajo Futuro	58
	Referencias	60

1 Introducción

1.1 Antecedentes y Significado de la Investigación

Actualmente, la nueva revolución de las tecnologías de la información, representada por el Internet de las Cosas (IoT, por sus siglas en inglés), está reconfigurando el modelo de funcionamiento social. La implementación de aplicaciones como ciudades inteligentes, automatización industrial, agricultura de precisión y hogares inteligentes depende del monitoreo continuo y el análisis inteligente de los parámetros físicos del entorno. Indicadores como la temperatura, humedad, calidad del aire, intensidad de luz, presión atmosférica y nivel de ruido no solo afectan la salud y confort individual, sino que también impactan la eficiencia energética en escenarios industriales. Por lo tanto, el desarrollo de soluciones de monitoreo ambiental inteligente con funcionamiento estable, costos controlables y despliegue flexible demuestra un valor significativo tanto en la dimensión de servicios sociales como en la económica.

Los sistemas de computación embebida constituyen la base tecnológica de los nodos periféricos del IoT, responsables de la adquisición, preprocesamiento y transmisión de datos de sensores. La línea de productos STM32 de STMicroelectronics, especialmente el modelo STM32F103C8T6 equipado con el núcleo Cortex-M3, ha obtenido una alta tasa de adopción en la enseñanza universitaria y la práctica de ingeniería debido a su buen equilibrio entre rendimiento computacional y consumo de energía, la diversidad de recursos de periféricos integrados y la madurez de la cadena de herramientas de desarrollo. Este chip ha acumulado una rica experiencia de aplicación en subcampos como el monitoreo ambiental, control industrial y electrónica de consumo.

Sin embargo, tanto en escenarios educativos como en el autoaprendizaje, la introducción y el avance en sistemas embebidos enfrentan algunos obstáculos prácticos:

(1) **Altos costos de hardware.** Un sistema completo de monitoreo ambiental requiere la adquisición de placas de desarrollo STM32, diversos módulos de sensores, depuradores, fuentes de alimentación y cables de conexión, con costos por conjunto típicamente entre varios cientos y miles de yuanes. Para los laboratorios universitarios, equipar decenas de conjuntos completos de equipos experimentales requiere una gran inversión de capital; para los estudiantes individuales, los altos costos de entrada también representan una barrera significativa.

(2) **Proceso de depuración complejo.** La depuración de sistemas embebidos depende del estado real de las conexiones del circuito de hardware; cualquier cable suelto, resistencia soldada incorrectamente o error en la configuración de un pin puede hacer que el sistema no funcione correctamente. Los principiantes, al enfrentar fallas de hardware complejas, a menudo tienen dificultades para localizar rápidamente la causa raíz del problema, consumiendo mucho tiempo en la resolución de problemas de hardware en lugar de

concentrarse en el aprendizaje de algoritmos de software y la lógica del sistema.

(3) **Alta barrera de comprensión de sensores.** Los principios de funcionamiento, protocolos de comunicación y métodos de procesamiento de datos de los diversos sensores varían considerablemente. Por ejemplo, el DHT11 utiliza un protocolo de bus único para transmitir 40 bits de datos, el MQ-135 requiere la adquisición de voltaje a través del ADC y la inferencia de la concentración de gases basada en modelos matemáticos complejos, y el BMP280 involucra comunicación I2C, lectura de ADC de 24 bits y algoritmos de compensación de temperatura. Comprender en profundidad los mecanismos de funcionamiento de estos sensores requiere que los estudiantes posean conocimientos en circuitos analógicos, circuitos digitales, protocolos de comunicación y procesamiento de datos.

(4) **Visualización de datos inconveniente.** En los entornos tradicionales de desarrollo embebido, los datos de los sensores generalmente se envían en formato de texto a través del puerto serie, y los desarrolladores necesitan ver los valores originales en asistentes de puerto serie, careciendo de medios intuitivos de visualización de datos. Incluso con software de host, a menudo se requiere trabajo de desarrollo adicional, y las funciones son relativamente limitadas, dificultando la implementación de funciones avanzadas como acceso remoto Web, consulta de datos históricos y alertas inteligentes.

Basado en el análisis anterior, el desarrollo de un sistema de virtualización de una estación inteligente de monitoreo ambiental basada en STM32, que permita a los estudiantes completar todo el proceso de desarrollo embebido sin necesidad de adquirir hardware físico, tiene un valor dual: por un lado, puede proporcionar una plataforma experimental de bajo costo para los planes de estudio; por otro lado, puede ofrecer un entorno de iteración rápida para la verificación temprana de conceptos de producto.

1.2 Estado Actual de la Investigación Nacional e Internacional

1.2.1 Investigación en Plataformas de Simulación Embebida

La investigación en virtualización de procesadores y simulación de sistemas embebidos comenzó relativamente temprano, y tanto la academia como la industria han invertido esfuerzos considerables. QEMU (Quick Emulator), como emulador de máquinas de código abierto, es compatible con múltiples arquitecturas de conjuntos de instrucciones, incluyendo la serie ARM Cortex-M. Su mecanismo central es la traducción dinámica de binarios, que puede convertir eficientemente el flujo de instrucciones de la plataforma objetivo al flujo de instrucciones del host, por lo que se utiliza frecuentemente en la portación de sistemas operativos y la verificación de software embebido. Sin embargo, el enfoque de simulación de QEMU se centra en el comportamiento del núcleo de CPU y del sistema operativo, y su soporte de modelado para periféricos integrados en microcontroladores (GPIO, ADC, TIM, etc.) aún no es completo, y su proceso de configuración y curva de aprendizaje no son amigables para escenarios de enseñanza y desarrollo rápido de prototi-

pos.

Proteus, desarrollado por Labcenter Electronics del Reino Unido, es una herramienta de automatización de diseño electrónico (EDA) que integra simulación de circuitos, diseño de PCB y depuración conjunta de microcontroladores. Este software es compatible con la simulación de chips de la serie STM32 e integra instrumentos virtuales y osciloscopios como medios de análisis auxiliar, siendo ampliamente utilizado en cursos de electrónica de instituciones de educación superior. Sin embargo, Proteus utiliza un modelo de licenciamiento comercial con costos elevados, y su velocidad de simulación está restringida por la precisión a nivel de circuito, mostrando una disminución significativa en la eficiencia de ejecución cuando la escala del sistema aumenta.

A nivel nacional, universidades como Tsinghua y Zhejiang han llevado a cabo exploraciones prospectivas en torno a entornos de experimentación virtual embebida. Algunas instituciones han construido plataformas de experimentación remota embebida basadas en Web, donde los estudiantes pueden acceder a hardware real desplegado en salas de computación a través del navegador para completar la programación de código y la depuración en línea. Aunque estos esquemas logran la compartición en red de recursos de hardware, el nivel inferior aún está vinculado a dispositivos físicos, sin eliminar fundamentalmente los costos de adquisición de hardware ni las limitaciones en la cantidad de dispositivos.

En los últimos años, el lenguaje Python ha experimentado un aumento continuo en su penetración en la investigación y la enseñanza, y las plataformas de enseñanza embebida basadas en simulación de software puro mediante lenguajes de alto nivel han comenzado a llamar la atención de los investigadores. Estos esquemas utilizan Python para describir el comportamiento del núcleo del MCU y los periféricos, con ventajas como ciclos de desarrollo cortos, fácil expansión de funciones y ejecución multiplataforma, pero aún existen brechas en la precisión de tiempos y la eficiencia de ejecución en comparación con emuladores de bajo nivel.

1.2.2 Investigación en Sistemas de Monitoreo Ambiental

En cuanto a sensores y equipos de monitoreo, los fabricantes internacionales han lanzado múltiples generaciones de productos comerciales. Los sensores de temperatura y humedad de la serie SHT de Sensirion (Estados Unidos), los sensores de presión de la serie BMP de Bosch (Alemania) y los sensores de gas de la serie CCS de AMS (Austria) representan los niveles avanzados de la industria. Estos productos generalmente poseen características como alta precisión de detección, bajo consumo de energía y interfaces digitales estandarizadas, siendo ampliamente adoptados por instrumentos de monitoreo ambiental de nivel profesional.

Las instituciones de investigación nacionales también han logrado progresos notables en esta dirección. Unidades de investigación como la Academia China de Ciencias y la Universidad Tsinghua han llevado a cabo investigaciones sistemáticas en torno al monitoreo

de contaminantes atmosféricos, detección de calidad del agua y medición de parámetros del suelo, formando varios dispositivos de monitoreo con propiedad intelectual independiente. En el mercado de consumo masivo, los dispositivos de detección ambiental para hogares inteligentes lanzados por empresas terminales como Xiaomi y Huawei generalmente integran múltiples tipos de elementos sensores y módulos de comunicación Wi-Fi, permitiendo a los usuarios consultar indicadores ambientales en tiempo real a través de aplicaciones móviles, con una alta aceptación en el mercado.

En cuanto a plataformas en la nube, plataformas como ThingSpeak y Adafruit IO en el extranjero, y OneNET y la plataforma IoT de Alibaba Cloud en el país, proporcionan infraestructura relativamente completa para el almacenamiento remoto, análisis computacional y presentación gráfica de datos de monitoreo. Estas plataformas generalmente utilizan el protocolo MQTT como medio de acceso de terminales e incluyen motores de reglas y mecanismos de notificación de eventos.

Es importante señalar que los resultados de investigación y productos comerciales existentes se orientan principalmente a escenarios de despliegue real, prestando insuficiente atención a las leyes de cognición y necesidades experimentales propias del proceso de enseñanza. Cómo abstraer y virtualizar el flujo de desarrollo de sistemas de monitoreo ambiental para adaptarlos a los objetivos de enseñanza de cursos de sistemas embebidos sigue siendo un tema que requiere una exploración más profunda.

1.3 Trabajo Principal de la Tesis

Esta tesis toma la virtualización de la estación inteligente de monitoreo ambiental STM32 como tema central, desarrollando la investigación desde las siguientes cuatro dimensiones:

(1) **Diseño e implementación de la plataforma de hardware virtual STM32.** Tomando el STM32F103C8T6 como prototipo, se diseñó un conjunto completo de plataforma de hardware virtual MCU, incluyendo la simulación del núcleo Cortex-M3, el sistema de mapeo de memoria, el temporizador SysTick, GPIO (entrada/salida de propósito general), ADC (convertidor analógico-digital), controlador de bus I2C, comunicación UART y temporizadores TIM. Esta plataforma está implementada en el lenguaje Python, con ventajas como alta legibilidad del código, fácil expansión y ejecución multiplataforma.

(2) **Desarrollo de controladores virtuales de sensores ambientales.** Basado en la plataforma de hardware virtual, se desarrollaron controladores virtuales para cinco tipos comunes de sensores ambientales: DHT11, MQ-135, BH1750, BMP280 y sensor de ruido. Se estableció un marco de modelado de sensores unificado, incluyendo definición de parámetros ambientales, modelos de conversión de cantidades físicas y modelos de ruido gaussiano, de modo que los datos simulados siguen las leyes físicas y poseen cierta aleatoriedad, acercándose más a las características de salida de sensores reales.

(3) **Construcción del sistema de visualización de datos Web.** Se desarrolló un

servicio backend utilizando el framework Python Flask, empleando la base de datos SQLite para almacenar datos de sensores, registros de alertas y registros del sistema, y diseñando interfaces API RESTful para su consumo por el frontend. El frontend utiliza tecnologías nativas HTML/CSS/JavaScript, combinado con la biblioteca de gráficos Chart.js para implementar la visualización de gráficos de líneas en tiempo real de datos de sensores, resumen de información estadística y funciones de panel de alertas.

(4) **Mecanismo de control e interacción de parámetros ambientales.** Se diseñó un mecanismo de comunicación entre procesos (IPC) basado en archivos compartidos, que implementa la comunicación bidireccional entre el panel de control Web y el MCU virtual a través del archivo `env_control.txt`. Los usuarios pueden ajustar los parámetros ambientales en tiempo real (como temperatura, humedad, presión, etc.) a través de la interfaz Web, y los sensores virtuales generan datos simulados correspondientes según los parámetros actualizados, logrando una interacción de circuito cerrado.

1.4 Estructura de la Tesis

El texto completo consta de siete capítulos, organizados de la siguiente manera:

El Capítulo 1 es la introducción, que presenta los antecedentes y significado de la investigación, el estado actual de la investigación nacional e internacional, el trabajo principal de la tesis y la organización de la estructura de la tesis.

El Capítulo 2 es el diseño general del sistema, que analiza los requisitos funcionales y no funcionales del sistema, diseña la arquitectura general de cuatro capas e introduce la selección de tecnologías clave.

El Capítulo 3 es el diseño de la plataforma de hardware virtual STM32, que describe en detalle los principios de simulación y métodos de implementación del núcleo virtual MCU, el sistema de memoria y diversos periféricos.

El Capítulo 4 es el controlador de sensores y la adquisición de datos, que introduce el marco de modelado general para la virtualización de sensores y analiza en detalle los principios de simulación y la implementación de controladores de cada sensor.

El Capítulo 5 es el diseño e implementación del sistema de software, incluyendo el diseño e implementación del módulo de almacenamiento de datos, sistema de alertas, sistema de visualización Web y panel de control.

El Capítulo 6 es la prueba y verificación del sistema, que realiza pruebas funcionales, pruebas de rendimiento y verificación de precisión de sensores, y presenta el análisis de los resultados de las pruebas.

El Capítulo 7 es el resumen y las perspectivas, que resume el trabajo principal de esta tesis y propone direcciones de mejora futuras.

2 Diseño General del Sistema

2.1 Análisis de Requisitos Funcionales

2.1.1 Requisitos Funcionales

Este sistema necesita implementar una plataforma de virtualización de una estación inteligente de monitoreo ambiental, cuyos objetivos funcionales abarcan los siguientes seis aspectos:

(1) **Simulación de MCU virtual:** El sistema debe ser capaz de simular las funciones principales del microcontrolador STM32F103C8T6, incluyendo la ejecución de instrucciones del procesador, acceso a memoria, sistema de reloj y control de periféricos. El código embebido en C escrito por el usuario (o código Python equivalente) debe poder ejecutarse correctamente en la plataforma virtual y migrarse al hardware real sin modificaciones.

(2) **Adquisición de datos de múltiples sensores:** El sistema debe soportar la adquisición de datos de al menos cinco tipos de sensores ambientales, incluyendo temperatura, humedad, calidad del aire (expresada como concentración de gases nocivos), intensidad de luz, presión atmosférica y nivel de ruido. Cada sensor debe tener un controlador independiente que siga el protocolo de comunicación correspondiente.

(3) **Almacenamiento y gestión de datos:** El sistema debe ser capaz de almacenar persistentemente los datos de sensores adquiridos en una base de datos, soportando funciones de consulta por rango de tiempo, análisis estadístico y exportación de datos. Al mismo tiempo, debe registrar los registros de operación del sistema y eventos de alerta para facilitar la resolución de problemas y auditorías.

(4) **Visualización de datos en tiempo real:** El sistema debe proporcionar una interfaz de visualización de datos en tiempo real basada en Web, mostrando de forma intuitiva las lecturas actuales e históricas de cada sensor en forma de tarjetas numéricas y gráficos de líneas. Los gráficos deben soportar funciones de zoom en el eje de tiempo y tooltips en los puntos de datos.

(5) **Función de alertas inteligentes:** El sistema debe soportar la configuración de umbrales superior e inferior para cada sensor, activando automáticamente alertas cuando las lecturas excedan el rango de umbrales, y registrando la información de alertas en la base de datos y el panel frontal. El estado de las alertas debe poder actualizarse y borrarse en tiempo real.

(6) **Control de parámetros ambientales:** El sistema debe permitir que los usuarios ajusten los parámetros ambientales en tiempo real a través de la interfaz Web, simulando el comportamiento de los sensores bajo diferentes condiciones ambientales. Las instrucciones de control deben surtir efecto oportunamente y retroalimentar los valores actuales de

configuración ambiental en la interfaz.

2.1.2 Requisitos No Funcionales

Además de los objetivos funcionales, el funcionamiento del sistema debe cumplir con las siguientes restricciones:

(1) **Requisitos de rendimiento:** La eficiencia de ejecución de instrucciones del MCU virtual debe ser suficientemente alta, con el tiempo de procesamiento de un solo ciclo de adquisición de datos no excediendo 1 segundo. El tiempo de respuesta de la API Web debe controlarse dentro de 100 milisegundos, y la frecuencia de actualización de los gráficos del frontend no debe ser inferior a 1 Hz.

(2) **Requisitos de confiabilidad:** El sistema debe ser capaz de funcionar de manera estable y continua durante al menos 72 horas sin fallos. Las operaciones de base de datos deben contar con mecanismos de protección de transacciones para prevenir la pérdida o inconsistencia de datos. Los controladores de sensores deben ser capaces de manejar excepciones como tiempos de espera de comunicación y errores de verificación.

(3) **Requisitos de escalabilidad:** La arquitectura del sistema debe tener un diseño modular adecuado. Al agregar nuevos tipos de sensores o módulos de periféricos, debe poder lograrse mediante la adición de archivos de controladores y archivos de configuración sin necesidad de modificar el código principal. El diseño de API del backend Web debe seguir las especificaciones RESTful para facilitar la expansión de funciones del frontend.

(4) **Requisitos de usabilidad:** El sistema debe proporcionar documentación e instrucciones de uso claras. La interfaz Web debe tener un diseño razonable y ser intuitiva de operar. Las interfaces de API del hardware virtual deben diseñarse de manera concisa para reducir la curva de aprendizaje y la dificultad de desarrollo para los usuarios.

(5) **Requisitos multiplataforma:** El sistema debe poder ejecutarse en los sistemas operativos principales como Windows, Linux y macOS, sin depender de características específicas de hardware o sistema operativo. La plataforma virtual debe implementarse en el lenguaje Python, aprovechando las características multiplataforma de Python para garantizar la compatibilidad.

2.2 Diseño de la Arquitectura General

Basado en los requisitos mencionados anteriormente, este sistema se divide de abajo hacia arriba en cuatro capas: capa de abstracción de hardware, capa de controladores de dispositivos, capa de lógica de negocio y capa de presentación de aplicaciones. Los límites funcionales de cada capa se muestran en la Tabla 1.

Cuadro 1: Diseño de la arquitectura de cuatro capas del sistema

Capa	Nombre	Descripción de Responsabilidades
Capa 4	Capa de Presentación de Aplicaciones	Responsable del renderizado de la interfaz Web frontend, dibujo de gráficos, interacción del usuario y visualización de datos, implementado con HTML/CSS/JavaScript nativo
Capa 3	Capa de Lógica de Negocio	Responsable del servicio backend Flask, diseño de API RESTful, gestión de almacenamiento de datos, procesamiento de lógica de alertas y análisis de instrucciones de control
Capa 2	Capa de Controladores de Dispositivos	Responsable del desarrollo de controladores de diversos sensores y la implementación del ciclo de adquisición de datos, convirtiendo cantidades físicas en cantidades digitales
Capa 1	Capa de Abstracción de Hardware	Responsable de la simulación del núcleo virtual MCU, sistema de memoria, controladores de bus y diversos periféricos, proporcionando interfaces a nivel de registro de hardware

La capa de abstracción de hardware (Hardware Abstraction Layer, HAL) constituye la base de toda la plataforma, con un comportamiento a nivel de registro consistente con el chip STM32 real. La capa de controladores de dispositivos invoca las interfaces expuestas por el HAL para completar la inicialización de sensores, lectura de datos y análisis de protocolos. La capa de lógica de negocio asume las tareas de procesamiento de datos brutos, almacenamiento persistente y análisis, y abre capacidades a la capa superior en forma de API estandarizada. La capa de presentación de aplicaciones se enfoca en la optimización de la calidad de presentación de la interfaz y la fluidez de la interacción.

Las capas colaboran a través de interfaces contractuales estandarizadas, y los módulos de capas superiores están prohibidos de operar directamente el estado interno de capas inferiores sin pasar por las interfaces. Este diseño controla eficazmente el grado de acoplamiento entre módulos. Cuando se necesita introducir un nuevo tipo de sensor, solo es necesario agregar el archivo de controlador correspondiente en la capa de controladores de dispositivos y registrar las reglas de análisis en la capa de lógica de negocio; tanto la capa de abstracción de hardware como la capa de presentación de aplicaciones no requieren ajustes.

2.3 Selección de Tecnologías

En la selección de herramientas y frameworks, este sistema sigue el criterio de “simple, eficiente y fácilmente extensible”. La combinación de tecnologías principales se resume en la Tabla 2.

Cuadro 2: Selección de tecnologías del sistema

Categoría	Tecnología	Razón de Selección
Lenguaje de desarrollo	Python 3.11	Sintaxis concisa, alta eficiencia de desarrollo, rica biblioteca estándar y bibliotecas de terceros, buen soporte multiplataforma
Framework Web	Flask 3.0	Ligero, flexible, fácil de aprender, comunidad activa, extensiones ricas
Base de datos	SQLite 3	Configuración cero, almacenamiento en archivo único, sin necesidad de proceso de servidor independiente, adecuado para aplicaciones pequeñas y medianas
Tecnología frontend	HTML/CSS/JS nativo	Sin dependencias de construcción, carga rápida, buena compatibilidad, fácil de desplegar
Biblioteca de gráficos	Chart.js 4.x	Código abierto gratuito, API amigable, soporte de diseño responsivo y efectos de animación
Control de versiones	Git	Gestión de versiones distribuida, gestión de ramas flexible, alta eficiencia de colaboración

La elección de Python 3.11 como lenguaje de desarrollo base se basa en tres consideraciones: primero, las reglas de sintaxis de Python son concisas y la legibilidad del código fuente es destacada, facilitando la explicación línea por línea en escenarios de enseñanza; segundo, la biblioteca estándar cubre tareas comunes como comunicación de red, acceso a bases de datos, operaciones de archivos y programación concurrente, reduciendo la introducción de dependencias externas; tercero, el modo de ejecución interpretado elimina el tiempo de espera de compilación, permitiendo la verificación inmediata después de modificaciones y acortando el ciclo de depuración.

Flask es un framework Web ligero basado en Python cuya filosofía enfatiza la simplicidad y flexibilidad. En comparación con frameworks full-stack como Django, Flask no impone estructuras de directorio de proyecto ni relaciones de vinculación de componentes

preestablecidas, permitiendo a los desarrolladores seleccionar bases de datos, motores de plantillas y estrategias de autenticación según las necesidades reales. Para una aplicación Web de escala moderada como la de este sistema, la cobertura funcional de Flask se encuentra en un rango razonable, evitando tanto el diseño excesivo como la falta de capacidades.

SQLite es una base de datos relacional embebida donde todos los datos se concentran en un único archivo de disco, sin necesidad de un proceso de servicio independiente. Esta propiedad de “listo para usar” simplifica enormemente el proceso de despliegue, siendo especialmente adecuada para desarrollo personal, experimentación de enseñanza y escenarios de aplicación a pequeña escala. SQLite es compatible con la sintaxis SQL estándar y transacciones ACID, suficiente para soportar los requisitos de este sistema en persistencia de datos y consultas analíticas.

La ruta de tecnología frontend seleccionó HTML5, CSS3 y JavaScript nativos, sin introducir frameworks componentizados como React o Vue. La motivación central de esta decisión es controlar la complejidad general, evitar la sobrecarga de configuración de la cadena de herramientas de construcción, y mantener el código fuente frontend en una forma directamente legible. Chart.js, como biblioteca de componentes de gráficos de código abierto, proporciona diversas categorías de gráficos y dimensiones de configuración, y su motor de renderizado basado en Canvas tiene un buen rendimiento, pudiendo satisfacer las demandas de rendimiento de la visualización de datos en tiempo real.

3 Diseño de la Plataforma de Hardware Virtual STM32

La plataforma de hardware virtual STM32 constituye el pilar inferior de todo el sistema. La idea de construcción de esta plataforma es reproducir el comportamiento a nivel de registro del microcontrolador STM32F103C8T6 en un entorno de software puro, de modo que los controladores de sensores y el código de la capa de aplicación puedan ejecutarse como lo harían en un chip real. A continuación se explican específicamente la simulación del núcleo del MCU, el mecanismo de mapeo de memoria y la simulación de los principales periféricos.

3.1 Diseño del Núcleo Virtual MCU

3.1.1 Resumen de Parámetros del STM32F103C8T6

El STM32F103C8T6 es producido por STMicroelectronics, equipado con el núcleo ARM Cortex-M3, y pertenece al modelo de densidad media de la serie STM32F1. Este chip logra un buen equilibrio entre rendimiento y costo, siendo ampliamente adoptado en la

enseñanza universitaria y la práctica de ingeniería. La Tabla 3 resume sus indicadores técnicos clave.

Cuadro 3: Parámetros técnicos principales del STM32F103C8T6

Parámetro	Especificación
Arquitectura del núcleo	ARM Cortex-M3
Frecuencia máxima	72 MHz
Memoria Flash	64 KB
SRAM	20 KB
Pines GPIO	37 (encapsulado LQFP48)
ADC	2 ADC de 12 bits, 16 canales
Temporizadores	3 temporizadores generales + 1 temporizador avanzado + 2 temporizadores
Interfaces de comunicación	2 I2C, 3 USART, 2 SPI, 1 CAN
Voltaje de operación	2.0 V – 3.6 V
Temperatura de operación	−40°C – +85°C
Forma de encapsulado	LQFP48

Dado que esta plataforma está implementada en Python y no realiza una simulación ciclo a ciclo del conjunto de instrucciones ARM o los ciclos de reloj, sino que se enfoca en la reproducción fiel del comportamiento de los registros de periféricos. La base de tiempo utiliza 1 milisegundo como unidad mínima, completando retardos y temporización con el módulo `time` de la biblioteca estándar de Python.

3.1.2 Simulación del Temporizador SysTick

El núcleo Cortex-M3 integra SysTick, un contador descendente de 24 bits que proporciona latidos periódicos al sistema operativo y las aplicaciones. En la plataforma virtual implementada en Python, la simulación de SysTick se enfoca en tres capacidades: decremento del contador, configuración del valor de recarga e interrupción por disparo. Limitado por las características de ejecución del lenguaje interpretado, la precisión de temporización a nivel de microsegundos es difícil de garantizar, por lo que se adopta un esquema aproximado basado en tiempo relativo.

La función `delay_ms` asume la responsabilidad del retardo a nivel de milisegundos. Esta función registra el tiempo actual del sistema en el punto de entrada, luego sondea y verifica el tiempo transcurrido, y retorna una vez que alcanza la cantidad de milisegundos establecida. Esta estrategia de espera activa ocupa la CPU, pero es suficiente para demostraciones de enseñanza y verificación funcional.

```
1 import time
2
3 def delay_ms(ms):
```

```
4      """
5      Función de retardo a nivel de milisegundos
6      Parametros:
7          ms: milisegundos de retardo
8      """
9      start = time.time()
10     while (time.time() - start) * 1000 < ms:
11         pass
```

Listing 1: Implementación de la función virtual delay_ms

El Código 1 presenta la implementación de espera activa de delay_ms. Durante el proceso de enseñanza, se puede organizar a los estudiantes para discutir los pros y contras de este esquema: su implementación es concisa y sin dependencias externas, lo cual es su ventaja; el hecho de que la CPU no pueda atender otras tareas durante el retardo es una clara desventaja. Si se desea mejorar, se puede intentar reescribirlo en una versión no bloqueante utilizando threading o asyncio.

3.2 Diseño del Sistema de Memoria

3.2.1 Modelo de Mapeo de Memoria

El STM32F103C8T6 asigna direcciones unificadas a la Flash, SRAM y registros de periféricos en un espacio de direcciones de 32 bits. El sistema de memoria virtual reproduce en consecuencia tres regiones de direcciones clave:

(1) **Memoria Flash:** Utilizada para almacenar el código del programa y datos constantes, con un rango de direcciones que comienza en 0x08000000 y un tamaño de 64 KB. En la plataforma virtual, la región de Flash se utiliza principalmente para almacenar código de firmware simulado y parámetros de configuración.

(2) **SRAM:** Utilizada para almacenar variables en tiempo de ejecución, pila y memoria heap, con un rango de direcciones que comienza en 0x20000000 y un tamaño de 20 KB. La región SRAM en la plataforma virtual se implementa como un arreglo de bytes, soportando operaciones de lectura y escritura de bytes, media palabra y palabra completa.

(3) **Registros de periféricos:** Incluyendo registros de configuración y datos de periféricos como GPIO, ADC, UART, TIM, I2C, con un rango de direcciones que comienza en 0x40000000. Cada periférico ocupa un espacio de direcciones continuo, con los registros dispuestos según desplazamientos fijos.

La clase MemoryRegion es el núcleo del sistema de memoria virtual, gestionando el flujo de bytes de lectura y escritura de un espacio de direcciones continuo. El STM32 adopta el modo little-endian, donde el byte bajo de datos multibyte se ubica en la dirección baja y el byte alto en la dirección alta. MemoryRegion sigue estrictamente esta regla de

orden de bytes durante el almacenamiento y acceso.

3.2.2 Implementación del Modo Little-Endian

El significado del modo Little-Endian es: el byte más significativo de los datos se almacena en la dirección alta, y el byte menos significativo se almacena en la dirección baja. Tomando 0x12345678 como ejemplo, su distribución de bytes en memoria es 0x78, 0x56, 0x34, 0x12 en orden, con las direcciones aumentando de baja a alta.

El módulo struct de Python proporciona medios de control de orden de bytes: el carácter de formato '<' indica little-endian y '>' indica big-endian. La clase MemoryRegion utiliza struct.pack y struct.unpack para completar la serialización y deserialización de datos multibyte, garantizando así que la plataforma virtual se comporte consistentemente con el STM32 real en el procesamiento del orden de bytes.

3.2.3 Mapeo de Registros

RegisterMap proporciona una capa adicional de encapsulamiento sobre MemoryRegion, orientada a la gestión de registros de periféricos. Cada instancia de periférico contiene un objeto RegisterMap, y los desarrolladores acceden a los registros por nombre en lugar de por dirección directa, mejorando así la legibilidad del código.

RegisterMap proporciona principalmente cuatro capacidades:

(1) **Registro de registros:** Define todos los registros que posee el periférico durante la inicialización, incluyendo nombre del registro, dirección de desplazamiento, valor por defecto y permisos de acceso (solo lectura / solo escritura / lectura-escritura).

(2) **Acceso a campos de bits:** Soporta operaciones de lectura y escritura de bits individuales o campos de bits dentro de los registros. Por ejemplo, se puede poner a 1 el bit 0 del registro CR1 mediante `set_bit('CR1', 0)`, sin necesidad de leer primero todo el registro y calcular manualmente.

(3) **Protección de escritura:** Para registros de solo lectura (como el registro de estado SR), RegisterMap rechazará operaciones de escritura directas, evitando que el código de capas superiores modifique accidentalmente valores de registros que no deberían modificarse.

(4) **Callback de cambio:** Permite registrar funciones callback para registros específicos, que se activan automáticamente cuando el valor del registro cambia. Este mecanismo es muy importante para simular la respuesta de comportamiento de los periféricos; por ejemplo, cuando se escribe en el registro ODR del GPIO, es necesario actualizar inmediatamente el estado de salida del pin.

3.3 Simulación de Periféricos

3.3.1 GPIO Entrada/Salida de Propósito General

El GPIO es responsable de la entrada y salida de señales digitales, siendo la primera puerta de interacción entre el microcontrolador y los circuitos externos. El GPIO del STM32F103C8T6 tiene 16 pines por grupo (Pin 0 a Pin 15), que pueden configurarse independientemente como entrada flotante, entrada pull-up, entrada pull-down, entrada analógica o salida push-pull / drenaje abierto.

El GPIO virtual se desarrolla en torno a los siguientes mecanismos:

(1) **Arreglo de estados de pines:** Cada puerto GPIO mantiene un arreglo de longitud 16, registrando el estado de nivel actual de cada pin (0 o 1). Para el modo de entrada, el estado del pin está determinado por dispositivos externos o resistencias pull-up/pull-down; para el modo de salida, el estado del pin está determinado por el valor del registro ODR.

(2) **Registro IDR de datos de entrada:** IDR (Input Data Register) se utiliza para leer el nivel de entrada actual de cada pin del puerto GPIO. En la plataforma virtual, el valor de IDR se obtiene mediante la codificación del arreglo de estados de pines. Específicamente, el bit i de IDR es igual al estado de nivel actual del pin i .

La fórmula de cálculo de IDR es:

$$\text{IDR} = \sum_{i=0}^{15} P_i \times 2^i \quad (1)$$

donde P_i representa el estado de nivel del pin i , $P_i \in \{0, 1\}$.

(3) **Registro ODR de datos de salida:** ODR (Output Data Register) se utiliza para establecer el nivel de salida de cada pin del puerto GPIO. Cuando se escribe un nuevo valor en ODR, el módulo GPIO virtual analiza bit por bit, actualiza el estado de salida del pin correspondiente y activa las funciones callback registradas.

(4) **Mecanismo de callback:** Para soportar la respuesta de dispositivos externos a los cambios de estado del GPIO, el GPIO virtual implementa un mecanismo de registro de callbacks. Los usuarios pueden registrar funciones callback para eventos de flanco ascendente, flanco descendente o cambio de nivel en pines específicos. Cuando el estado del pin cambia de manera correspondiente, todas las funciones callback registradas se ejecutan en orden de registro. Este mecanismo es muy útil al simular la salida de interrupción de sensores y el control de encendido/apagado de indicadores LED.

3.3.2 ADC Convertidor Analógico-Digital

El ADC mapea voltajes analógicos continuos a cantidades digitales discretas, actuando como traductor entre el frontend analógico y el backend digital. El STM32F103C8T6 integra dos ADC de tipo SAR de 12 bits, soportando 16 canales externos y 2 canales

internos (sensor de temperatura interno y voltaje de referencia).

Los atributos clave del ADC virtual son los siguientes:

(1) **Resolución de 12 bits:** El resultado de conversión del ADC es un entero sin signo de 12 bits, con un rango de valores de 0 a 4095. La fórmula de cuantización correspondiente es:

$$D = \left\lfloor \frac{V_{in}}{V_{ref}} \times 4096 \right\rfloor \quad (2)$$

donde D es la salida digital, V_{in} es el voltaje analógico de entrada, y V_{ref} es el voltaje de referencia (típicamente 3.3 V).

(2) **Conversión voltaje–digital:** Cuando el código de capa superior lee un canal a través del ADC, el ADC virtual calcula la cantidad digital correspondiente según el valor de voltaje analógico actualmente conectado a ese canal, siguiendo la fórmula 2. El valor de voltaje analógico es proporcionado por el sensor virtual conectado o la fuente de señal externa.

(3) **Modelo de ruido:** Para simular más fielmente las características de conversión del ADC, se superpone ruido blanco gaussiano sobre el resultado de conversión ideal. La amplitud del ruido puede ajustarse mediante parámetros de configuración, siendo el valor por defecto ± 2 LSB (Least Significant Bit). La fórmula de conversión con ruido es:

$$D_{actual} = D_{ideal} + N(0, \sigma^2) \quad (3)$$

donde $N(0, \sigma^2)$ representa un número aleatorio con distribución normal de media 0 y varianza σ^2 , y el valor por defecto de σ es aproximadamente 2.

(4) **Selección y habilitación de canales:** El ADC virtual soporta la configuración de múltiples canales, definiendo el orden y cantidad de canales de conversión mediante el registro de secuencia SQR (Regular Sequence Register). Antes de realizar la conversión, es necesario habilitar primero el ADC (poner a 1 el bit ADON del registro CR2), y luego iniciar la conversión (poner a 1 el bit SWSTART del registro CR2).

3.3.3 Controlador de Bus I2C

El bus I2C completa la comunicación multi-maestro multi-esclavo mediante dos líneas SDA y SCL, siendo ampliamente utilizado en dispositivos como sensores y EEPROM. El STM32F103C8T6 está equipado con dos interfaces I2C, compatibles con el modo estándar (100 kHz) y el modo rápido (400 kHz).

El bus I2C virtual se enfoca en simular los siguientes comportamientos:

(1) **Montaje de dispositivos:** El bus I2C virtual mantiene una lista de dispositivos montados, donde cada dispositivo se identifica únicamente por una dirección de esclavo de 7 bits. Cuando el código de capa superior envía una señal de inicio y un byte de dirección a través del I2C, el controlador de bus busca la instancia del dispositivo virtual correspondiente según la dirección, estableciendo la conexión de comunicación.

La definición de la interfaz de montaje de dispositivos es la siguiente:

```
1 def attach_device(self, address, device):
2     """
3     Monta un dispositivo virtual en el bus I2C
4     Parametros:
5         address: direccion de esclavo de 7 bits
6         device: instancia de dispositivo virtual, debe implementar
7                 metodos read/write
8     """
9     self.devices[address] = device
```

Listing 2: Interfaz de montaje de dispositivos virtuales I2C

(2) **Operaciones de lectura y escritura:** El I2C virtual soporta operaciones de lectura y escritura a nivel de byte y a nivel de búfer. En operaciones de escritura, el dispositivo maestro envía la dirección del esclavo y el bit de escritura (0), seguido de la dirección del registro a escribir y los bytes de datos; en operaciones de lectura, el dispositivo maestro primero envía la dirección del esclavo y el bit de escritura para especificar la dirección del registro, luego reenvía la señal de inicio junto con la dirección del esclavo y el bit de lectura (1), y el dispositivo esclavo devuelve los bytes de datos solicitados.

(3) **Manejo de errores:** El I2C virtual implementa mecanismos básicos de detección de errores, incluyendo: sin respuesta de dirección (NACK) — cuando la dirección del esclavo enviada no encuentra una coincidencia en la lista de dispositivos, se devuelve un error; bus ocupado — cuando múltiples dispositivos maestros intentan controlar el bus simultáneamente, se produce una detección de conflicto; verificación de datos — soporte opcional de verificación CRC de los datos transmitidos.

3.3.4 Comunicación UART Puerto Serie

UART, como interfaz serial asíncrona, se utiliza comúnmente para el intercambio de datos entre el MCU y dispositivos como PC, módulos Bluetooth y receptores GPS. El STM32F103C8T6 posee tres USART independientes, soportando comunicación full-duplex y múltiples configuraciones de baudios.

El UART virtual se enfoca en los siguientes puntos en su implementación:

(1) **Configuración de baudios:** La tasa de baudios representa la cantidad de bits transmitidos por segundo, siendo un parámetro clave en la comunicación UART. El UART virtual registra el valor de baudios configurado actualmente; aunque debido a la naturaleza de la simulación por software no puede controlar realmente la temporización a nivel de bit según la tasa de baudios, el valor de baudios afecta ciertos comportamientos de simulación relacionados con la temporización (como la configuración de tiempo de espera). Los valores comunes de baudios incluyen 9600, 115200, 921600, etc.

(2) **Simulación de redirección de printf:** En el desarrollo embebido, generalmente se redirige la función de salida estándar printf al UART para enviar información de depuración a través del puerto serie. El UART virtual proporciona una interfaz equivalente de envío de cadenas, almacenando la cadena a enviar en un búfer interno y activando la función callback del receptor.

(3) **Mecanismo de callback:** El UART virtual soporta el registro de funciones callback de recepción, que se llaman automáticamente cuando llegan datos. Este mecanismo simula la interrupción de recepción del UART real, permitiendo que el código de capa superior procese los datos del puerto serie de manera dirigida por eventos, sin necesidad de sondear el búfer de recepción.

```
1 class VirtualUART:
2     def __init__(self):
3         self.baudrate = 115200
4         self.rx_callback = None
5         self.tx_buffer = []
6
7     def set_rx_callback(self, callback):
8         """Registra la funcion callback de recepcion"""
9         self.rx_callback = callback
10
11    def send(self, data):
12        """Envia datos (cadena o bytes)"""
13        if isinstance(data, str):
14            data = data.encode('utf-8')
15        self.tx_buffer.extend(data)
16        if self.rx_callback:
17            self.rx_callback(data)
18
19    def printf(self, fmt, *args):
20        """Salida formateada, simula printf"""
21        msg = fmt % args
22        self.send(msg)
```

Listing 3: Registro de callback y envio del UART virtual

El Código 3 presenta la implementación de la clase VirtualUART. La interfaz send es compatible simultáneamente con tipos str y bytes, mientras que printf reproduce la semántica de la salida formateada del lenguaje C.

3.3.5 TIM Temporizador General

TIM tiene las funciones más diversas en la familia de periféricos STM32, pudiendo actuar como temporizador, contador, generador de ondas PWM o capturar de entrada externa. Este proyecto solo utiliza su capacidad básica de temporización, proporcionando una fuente de disparo de frecuencia fija para el muestreo de sensores.

VirtualTIM utiliza hilos daemon del módulo threading de Python para completar la temporización en segundo plano. Los hilos daemon finalizan automáticamente cuando el hilo principal sale, sin necesidad de limpieza manual, siendo adecuados para las necesidades de tareas periódicas en segundo plano.

La lógica de operación del TIM virtual puede dividirse en tres pasos:

(1) **Incremento del contador:** Después de que el TIM se inicia, el hilo daemon incrementa el contador interno a intervalos de tiempo fijos (determinados por el coeficiente de preescala y el valor de recarga automática). La frecuencia de incremento del contador se calcula mediante la fórmula:

$$f_{count} = \frac{f_{clk}}{(PSC + 1) \times (ARR + 1)} \quad (4)$$

donde f_{clk} es la frecuencia de reloj del temporizador (típicamente 72 MHz), PSC es el valor del preescalador, y ARR es el valor de recarga automática.

(2) **Callback periódico:** Cuando el contador alcanza el valor de recarga automática, el contador se reinicia a cero y se activa el evento de actualización (Update Event). Si se ha registrado una función callback periódica, se llama en este momento. En el sistema de monitoreo ambiental, la función callback periódica generalmente ejecuta la tarea de adquisición de datos de sensores, logrando así el muestreo automático de frecuencia fija.

(3) **Inicio y parada:** El TIM se controla mediante el bit CEN del registro CR1 para iniciar y detener. Poner a 1 el bit CEN inicia el temporizador, y ponerlo a 0 lo detiene. El TIM virtual crea e inicia el hilo daemon al iniciar, y establece una bandera de evento para notificar al hilo que salga al detenerse.

```
1 import threading
2 import time
3
4 class VirtualTIM:
5     def __init__(self):
6         self.prescaler = 7199          # PSC
7         self.auto_reload = 9999        # ARR
8         self.counter = 0
9         self.running = False
10        self.period_callback = None
11        self._stop_event = threading.Event()
```



```
12
13 def set_period_callback(self, callback):
14     """Establece la funcion callback periodica"""
15     self.period_callback = callback
16
17 def _timer_thread(self):
18     """Hilo daemon del temporizador"""
19     period_ms = (self.prescaler + 1) * (self.auto_reload + 1) /
20                 72000
21     while not self._stop_event.is_set():
22         time.sleep(period_ms / 1000.0)
23         self.counter = 0
24         if self.period_callback:
25             self.period_callback()
26
27 def start(self):
28     """Inicia el temporizador"""
29     self.running = True
30     self._stop_event.clear()
31     t = threading.Thread(target=self._timer_thread, daemon=True)
32     t.start()
33
34 def stop(self):
35     """Detiene el temporizador"""
36     self.running = False
37     self._stop_event.set()
```

Listing 4: Implementacion del hilo daemon del TIM virtual

El Código 4 muestra cómo VirtualTIM utiliza hilos daemon para lograr el disparo periódico. El `period_callback` se ejecuta en cada desbordamiento de período. Modificar `prescaler` y `auto_reload` permite reconfigurar el período de temporización, adaptándose a diversas velocidades de muestreo.

4 Controladores de Sensores y Adquisición de Datos

Los sensores son equivalentes a los órganos sensoriales del sistema de monitoreo, transformando diversos parámetros del entorno físico en señales eléctricas para su procesamiento por el MCU. En un entorno de simulación de software puro, el modelo del sensor actúa como la capa de conexión entre el hardware virtual y la aplicación de capa superior,

y su grado de realismo impacta directamente en la credibilidad de todo el sistema. A continuación se discute primero el enfoque de modelado general, y luego se analiza individualmente los principios de simulación y la codificación de controladores de sensores específicos.

4.1 Principios de Virtualización de Sensores

4.1.1 Marco de Modelado General

Este sistema estableció un marco de modelado unificado para la simulación de sensores, reduciendo todos los tipos de sensores a tres eslabones: entrada de parámetros ambientales, modelo de conversión de cantidades físicas, y salida digital. La Figura ?? presenta el diagrama conceptual de este marco.

Este marco contiene tres elementos centrales:

(1) **Definición de parámetros ambientales:** Cada sensor se asocia con un conjunto de variables de parámetros ambientales, que representan los valores reales de las cantidades físicas medidas por el sensor. Por ejemplo, el sensor de temperatura se asocia con el parámetro *temperature*, y el sensor de luz se asocia con el parámetro *illuminance*. Los valores de los parámetros ambientales pueden ajustarse en tiempo real a través del panel de control, simulando diferentes condiciones ambientales.

(2) **Modelo de conversión de cantidades físicas:** Describe cómo los parámetros ambientales se convierten en la cantidad física de salida del sensor (típicamente voltaje, resistencia o cantidad digital). Los modelos de conversión varían según el sensor, pudiendo ser relaciones lineales, relaciones de potencia o funciones por tramos. Los modelos de conversión deben establecerse basándose en las hojas de datos (Datasheet) del sensor y los principios físicos, garantizando la razonabilidad de los datos simulados.

(3) **Modelo de ruido:** La salida de sensores reales inevitablemente contiene ruido aleatorio, cuyas fuentes incluyen ruido térmico, ruido de disparo e interferencia electromagnética. Para mejorar la autenticidad de la simulación, se superpone ruido blanco gaussiano sobre la salida ideal del sensor. La expresión matemática del modelo de ruido es:

$$Y = X + N(0, \sigma^2) \quad (5)$$

donde Y es el valor real de salida del sensor, X es el valor de salida ideal (calculado por el modelo de conversión de cantidades físicas), y $N(0, \sigma^2)$ es el ruido aleatorio con distribución normal de media 0 y varianza σ^2 .

La intensidad del ruido σ se determina según el grado de precisión del sensor y el escenario de aplicación típico. Para sensores de alta precisión (como el BMP280), σ toma un valor menor; para sensores de baja precisión (como el MQ-135), σ toma un valor mayor. En escenarios especiales, también se pueden agregar valores atípicos (Outlier) para simular

fallas del sensor o cambios abruptos en el entorno.

4.1.2 Interfaz del Controlador de Sensores

Los controladores de sensores virtuales comparten el mismo conjunto de contratos de interfaz, y el código de capa superior puede invocarlos sin preocuparse por las diferencias de nivel inferior. La definición de la interfaz es la siguiente:

- `init()`: Inicializa el sensor, configura los parámetros de comunicación y el modo de operación.
- `read()`: Lee el valor de medición actual del sensor, devolviendo datos brutos o cantidades físicas calibradas.
- `calibrate()`: Ejecuta la operación de calibración, actualizando los coeficientes de calibración (si aplica).
- `set_environment(env_dict)`: Actualiza los parámetros ambientales asociados al sensor.

Gracias a la consistencia de las interfaces, al introducir un nuevo tipo de sensor solo es necesario escribir la clase de controlador correspondiente e implementar los cuatro métodos, mientras que el ciclo de adquisición de datos y la capa de visualización Web permanecen sin cambios.

4.2 Sensor de Temperatura y Humedad DHT11

4.2.1 Descripción General del Sensor

El DHT11 integra la detección de temperatura y humedad en un único chip, comunicándose con el host mediante un protocolo de bus único. Su característica destacada es el bajo costo, tamaño compacto y fácil manejo, siendo muy común en proyectos introductorios de IoT. La Tabla 4 lista sus especificaciones principales.

Cuadro 4: Parámetros técnicos principales del DHT11

Parámetro	Especificación
Rango de medición (humedad)	20% – 90% RH
Precisión de medición (humedad)	±5% RH
Rango de medición (temperatura)	0°C – 50°C
Precisión de medición (temperatura)	±2°C
Resolución	1% RH / 1°C
Ciclo de muestreo	≥ 1 segundo
Voltaje de alimentación	3.3 V – 5.5 V
Protocolo de comunicación	Bus único serial

4.2.2 Simulación del Protocolo de Bus Único

La comunicación de bus único del DHT11 puede descomponerse en tres fases: inicio por el host, respuesta del sensor, y transmisión de datos bit por bit.

(1) **Inicio del host:** El MCU pone la línea de datos en bajo durante al menos 18 ms, luego la libera y espera la respuesta del DHT11. Esta acción se utiliza para despertar al sensor en estado de espera, indicándole que está a punto de enviar datos.

(2) **Respuesta del sensor:** Después de detectar el pulso de inicio, el DHT11 primero emite un nivel bajo de 80 μ s, seguido de un nivel alto de 80 μ s, para informar al host que está listo. El host confirma que el sensor está en línea capturando esta secuencia de pulsos.

(3) **Transmisión del marco de datos:** El DHT11 informa la temperatura y humedad en un formato de trama de 40 bits. Cada bit comienza con un nivel bajo de 50 μ s, seguido de un ancho de nivel alto que determina el valor lógico: 26–28 μ s representan “0”, y aproximadamente 70 μ s representan “1”. Los 40 bits se disponen secuencialmente como 8 bits de entero de humedad, 8 bits de decimal de humedad, 8 bits de entero de temperatura, 8 bits de decimal de temperatura y 8 bits de suma de verificación.

El controlador VirtualDHT11 reproduce completamente la secuencia de temporización del protocolo mencionado anteriormente. La implementación clave es la siguiente:

```
1 class VirtualDHT11:
2     def __init__(self, gpio, pin):
3         self.gpio = gpio
4         self.pin = pin
5         self.humidity = 50.0      # humedad predeterminada 50%
6         self.temperature = 25.0   # temperatura predeterminada 25C
7
8     def set_environment(self, env):
9         self.humidity = env.get('humidity', 50.0)
```

```
10     self.temperature = env.get('temperature', 25.0)
11
12     def read(self):
13         # Simula la senal de inicio
14         self.gpio.set_mode(self.pin, 'OUTPUT')
15         self.gpio.write(self.pin, 0)
16         delay_ms(20)
17         self.gpio.write(self.pin, 1)
18         delay_us(30)
19         self.gpio.set_mode(self.pin, 'INPUT')
20
21         # Simula la senal de respuesta y transmision de datos
22         # Retorna (humedad, temperatura)
23         h_int = int(self.humidity)
24         h_dec = int((self.humidity - h_int) * 10)
25         t_int = int(self.temperature)
26         t_dec = int((self.temperature - t_int) * 10)
27         checksum = (h_int + h_dec + t_int + t_dec) & 0xFF
28
29         data = [h_int, h_dec, t_int, t_dec, checksum]
30         return self.humidity + random.gauss(0, 1.5), \
31                self.temperature + random.gauss(0, 0.8)
```

Listing 5: Senal de inicio y lectura de datos del DHT11

En el Código 5, VirtualDHT11 reproduce la secuencia de temporización del bus único alternando los niveles del GPIO. En la fase de generación de datos, empaqueta los parámetros ambientales en una trama de 40 bits y adjunta la suma de verificación, luego inyecta ruido gaussiano según la fórmula 5: la desviación estándar del canal de humedad se establece en 1.5% RH, y la del canal de temperatura en 0.8°C, correspondiendo al grado de precisión del DHT11 real.

4.3 Sensor de Calidad del Aire MQ-135

4.3.1 Principio del Sensor

El MQ-135 muestra alta sensibilidad a múltiples gases como amoníaco, sulfuros, vapores de benceno y humo. La capa sensible utiliza una película delgada semiconductor de SnO₂; las moléculas de gas objetivo experimentan reacciones de oxidación-reducción en su superficie, causando cambios en la resistencia volumétrica que reflejan la concentración de gas.

Existe una relación de potencia entre la resistencia del elemento sensible R_S del MQ-

135 y la concentración de gas C :

$$\frac{R_S}{R_0} = A \cdot C^{-B} \quad (6)$$

donde R_0 es el valor de resistencia de referencia del sensor en aire limpio, A y B son constantes empíricas relacionadas con el tipo de gas, y C es la concentración de gas (en ppm). Para escenarios generales de monitoreo de calidad del aire, típicamente se toma $A \approx 116.6$, $B \approx 2.77$.

4.3.2 Inversión Voltaje–Concentración

El módulo MQ-135 utiliza internamente un circuito divisor de voltaje para convertir los cambios de resistencia en señales de voltaje. El voltaje de salida y la resistencia del elemento sensible satisfacen:

$$V_{out} = V_{cc} \cdot \frac{R_L}{R_S + R_L} \quad (7)$$

donde V_{cc} es el voltaje de alimentación (típicamente 5 V), y R_L es la resistencia de carga (potenciómetro ajustable).

A partir de esto se puede despejar la resistencia del elemento sensible:

$$R_S = R_L \cdot \left(\frac{V_{cc}}{V_{out}} - 1 \right) \quad (8)$$

Sustituyendo la expresión anterior en la fórmula 6, se obtiene la fórmula analítica de la concentración de gas:

$$C = \left(\frac{A \cdot R_0}{R_S} \right)^{\frac{1}{B}} = \left(\frac{A \cdot R_0 \cdot V_{out}}{R_L \cdot (V_{cc} - V_{out})} \right)^{\frac{1}{B}} \quad (9)$$

4.3.3 Implementación del Controlador Virtual

El controlador VirtualMQ135 toma el parámetro ambiental `air_quality` (en ppm) como entrada, y calcula inversamente el voltaje del ADC según las fórmulas 6 y 9. Además, el controlador reproduce tres comportamientos típicos del dispositivo real:

(1) **Simulación de precalentamiento:** El sensor MQ-135 requiere un período de precalentamiento después de ser energizado para alcanzar un estado de operación estable, siendo el tiempo típico de precalentamiento de 20 segundos. El controlador virtual registra el tiempo de inicio después de la inicialización, y durante el período de precalentamiento devuelve datos inválidos o datos de menor precisión.

(2) **Respuesta no lineal:** Según la relación de potencia, la concentración de gas y el valor de resistencia presentan una relación no lineal. En la región de baja concentración, la sensibilidad del sensor es mayor; en la región de alta concentración, la sensibilidad disminuye gradualmente. El controlador virtual simula completamente esta característica

no lineal.

(3) **Compensación de temperatura y humedad:** La salida del MQ-135 se ve afectada por la temperatura y humedad ambientales. El controlador virtual introduce coeficientes simples de compensación de temperatura y humedad, corrigiendo el valor de concentración según la temperatura y humedad actuales para mejorar la autenticidad de los datos simulados.

4.4 Sensor de Luz BH1750

4.4.1 Descripción General del Sensor

El BH1750 emite directamente una cantidad digital de iluminancia en unidades de lux a través del bus I2C, eliminando la necesidad de un ADC externo y una calibración tediosa. Su rango amplio, alta precisión y simple cableado lo hacen ampliamente desplegado en escenarios como iluminación automática, regulación de retroiluminación y detección ambiental.

El modo de medición del BH1750 está determinado por el byte de comando. La Tabla 5 resume las instrucciones comúnmente utilizadas.

Cuadro 5: Comandos I2C comunes del BH1750

Valor de comando	Nombre	Descripción de función
0x01	Power On	Comando de encendido, despierta al sensor
0x00	Power Down	Comando de apagado, entra en modo de bajo consumo
0x10	Continuous H-Res Mode	Modo continuo de alta resolución (1 lux de resolución, 120ms tiempo de medición)
0x11	Continuous H-Res2 Mode	Modo continuo de alta resolución 2 (0.5 lux de resolución, 120ms tiempo de medición)
0x13	Continuous L-Res Mode	Modo continuo de baja resolución (4 lux de resolución, 16ms tiempo de medición)
0x20	One Time H-Res Mode	Modo de alta resolución de una sola vez

4.4.2 Fórmula de Cálculo de Iluminancia

En el modo H-Res, el BH1750 emite un conteo raw de 16 bits, que multiplicado por un factor de sensibilidad de 1.2 lux/conteo da la iluminancia real:

$$E_v = \text{raw} \times 1.2 \quad (10)$$

donde E_v es la iluminancia (unidad: lux), y raw es el dato raw de 16 bits devuelto por el sensor.

Si el valor raw es 500, la iluminancia correspondiente es $500 \times 1.2 = 600$ lux.

VirtualBH1750, tras recibir comandos a través de I2C, calcula inversamente el conteo raw según el parámetro ambiental illuminance actual, siguiendo la fórmula 10:

$$\text{raw} = \left\lfloor \frac{E_v}{1.2} \right\rfloor + N(0, \sigma^2) \quad (11)$$

donde σ toma 3 unidades de conteo, correspondiendo a aproximadamente 3.6 lux de ruido de medición, consistente con las especificaciones de la hoja de datos del BH1750.

4.5 Sensor de Presión BMP280

4.5.1 Descripción General del Sensor

El BMP280, de Bosch, está basado en tecnología MEMS, combinando pequeño volumen, bajo consumo y alta precisión. Se puede encontrar en campos como estaciones meteorológicas, altímetros, navegación interior y estabilización de altura de drones. La Tabla 6 enumera los indicadores técnicos principales.

Cuadro 6: Parámetros técnicos principales del BMP280

Parámetro	Especificación
Rango de medición de presión	300 hPa – 1100 hPa
Precisión de medición de presión	± 0.12 hPa (valor típico)
Rango de medición de temperatura	$-40^\circ\text{C} - +85^\circ\text{C}$
Precisión de medición de temperatura	$\pm 0.5^\circ\text{C}$ (a 25°C)
Precisión absoluta	± 1 hPa
Interfaz de comunicación	I2C o SPI
ID del chip	0x58

4.5.2 Verificación del ID del Chip

El BMP280 tiene un registro de ID de chip de solo lectura en la dirección 0xD0, cuyo valor tras el reinicio es 0x58. Después del encendido, el host generalmente lee primero este

registro para confirmar que el dispositivo está en línea. VirtualBMP280 incorpora este comportamiento desde la fase de inicialización, y las lecturas de acceso a 0xD0 devuelven fijamente 0x58.

4.5.3 ADC de 24 bits y Coeficientes de Calibración

El BMP280 integra un ADC de 24 bits, que cuantiza respectivamente las salidas analógicas de los sensores de presión y temperatura. Los bits efectivos de los canales de presión y temperatura son ambos de 20 bits, empaquetados en registros de 3 bytes. Su extrema resolución le permite capturar fluctuaciones de presión a nivel de milibares.

El BMP280 real almacena los coeficientes de compensación en la NVM (direcciones 0x88–0xA1) durante la calibración de fábrica. El host debe extraer estos coeficientes antes de la lectura, y luego ejecutar en el MCU un algoritmo de compensación de temperatura que incluye términos cuadráticos y cúbicos, para restaurar los valores físicos a partir de los valores raw del ADC.

La plataforma virtual realiza una simplificación razonable del proceso de compensación, aproximando la relación entre presión, temperatura y valores raw del ADC mediante una relación lineal:

$$\text{raw_P} = \left\lfloor \frac{P}{1013.25} \times 415148 \right\rfloor + N(0, \sigma_P^2) \quad (12)$$

$$\text{raw_T} = \lfloor (T + 40) \times 100 \rfloor + N(0, \sigma_T^2) \quad (13)$$

donde P es el valor de presión ambiental (unidad: hPa), T es el valor de temperatura ambiental (unidad: °C), 415148 es el valor raw típico del ADC correspondiente a 1013.25 hPa (presión atmosférica estándar), y σ_P y σ_T son las desviaciones estándar del ruido.

La derivación de la fórmula 12 se basa en el hecho empírico de que, en el modo de sobremuestreo $\times 16$, el valor raw típico del ADC correspondiente a la presión atmosférica estándar de 1013.25 hPa es aproximadamente 415148. Aunque la relación de proporción presión–valor raw establecida de esta manera omite la compleja compensación de temperatura y corrección no lineal presente en el chip real, aún posee suficiente precisión para demostraciones de enseñanza y verificación general.

4.6 Sensor de Ruido

4.6.1 Principio del Sensor

Los sensores de sonido (comúnmente como el módulo KY-038 o basados en LM393) miden la intensidad del ruido ambiental, presentando el resultado en decibelios (dB). Estos módulos generalmente consisten en un micrófono de electret de alta sensibilidad y circuitos de acondicionamiento de señal, que convierten las ondas de presión acústica en voltaje analógico.

El decibelio es una unidad de relación logarítmica. El nivel de presión sonora SPL (Sound Pressure Level) se define mediante la siguiente fórmula:

$$L_p = 20 \cdot \log_{10} \left(\frac{p}{p_0} \right) \quad (14)$$

donde L_p es el nivel de presión sonora (unidad: dB), p es la presión acústica real (unidad: Pa), y p_0 es la presión de referencia (típicamente 20×10^{-6} Pa, es decir, el umbral auditivo del oído humano a 1 kHz).

4.6.2 Mapeo Lineal de Voltaje

Tras la amplificación y rectificación, el módulo del sensor emite un voltaje analógico proporcional a la presión acústica. En el modelo virtual, se asume adicionalmente una relación lineal entre el voltaje y el nivel de presión sonora:

$$V_{out} = V_{offset} + k \cdot L_p \quad (15)$$

donde V_{offset} es el voltaje de offset (correspondiente a la salida a 0 dB, que en la práctica generalmente es un valor positivo pequeño), y k es el coeficiente de sensibilidad (unidad: V/dB).

El controlador VirtualNoise lee el parámetro ambiental `noise_level` (en dB), calcula el voltaje de salida según la fórmula 15, y luego lo cuantiza a un valor digital mediante el ADC. Para acercarse a escenarios reales, la lectura final también se superpone con ruido gaussiano de desviación estándar 1.5 dB, comparable con la precisión típica de sonómetros de nivel de consumo.

VirtualNoise reproduce adicionalmente las siguientes tres características físicas:

(1) **Direccionalidad:** Los micrófonos reales poseen cierta directividad, siendo la sensibilidad diferente para sonidos provenientes de diferentes direcciones. El sensor virtual introduce un coeficiente de directividad simple para simular esta característica física.

(2) **Respuesta de frecuencia:** El oído humano tiene diferente sensibilidad para sonidos de diferentes frecuencias, siendo la ponderación A (A-weighting) la forma más común de ponderar el nivel de presión sonora. El sensor virtual emite por defecto el nivel de presión sonora ponderado A, acercándose más a la sensación subjetiva del oído humano.

(3) **Retención de pico:** Algunos sensores de ruido soportan la función de detección de picos, registrando el nivel máximo de presión sonora durante un período de tiempo. El sensor virtual implementa un modo opcional de retención de pico, registrando y devolviendo el valor máximo de dB dentro de una ventana de tiempo especificada.

4.7 Diseño del Ciclo de Adquisición de Datos

El ciclo de adquisición de datos impulsa el flujo principal de toda la estación virtual de monitoreo: invoca a cada sensor para completar el muestreo según un período fijo, y luego empuja los resultados hacia la cadena de almacenamiento y visualización. Un solo ciclo se descompone en 6 eslabones:

1. **Lectura de parámetros ambientales:** Lee la configuración actual de parámetros ambientales desde el archivo compartido `env_control.txt`, incluyendo temperatura, humedad, presión, iluminancia, calidad del aire y nivel de ruido.
2. **Actualización del entorno de sensores:** Pasa los parámetros ambientales leídos a cada instancia de sensor virtual, permitiendo que los sensores generen datos simulados según la última configuración ambiental.
3. **Ejecución de lectura de sensores:** Invoca secuencialmente el método `read()` de cada sensor para obtener los valores de medición actuales. El orden de lectura generalmente es: DHT11 (temperatura/humedad) → BMP280 (presión/temperatura) → BH1750 (iluminancia) → MQ-135 (calidad del aire) → sensor de ruido.
4. **Formateo de datos:** Convierte las lecturas brutas de cada sensor a unidades estándar (como lux, hPa, ppm, dB, etc.), y organiza los datos en formato de diccionario o JSON según una estructura de datos predefinida.
5. **Almacenamiento de datos:** Inserta los datos formateados en la tabla `sensor_data` de la base de datos SQLite, actualizando simultáneamente la caché de datos más recientes en memoria.
6. **Verificación de alertas:** Recorre la configuración de alertas de cada sensor, verificando si la lectura actual excede el rango de umbrales. En caso de excederse, registra el registro de alerta y actualiza el estado de alerta.

Para evitar que el tiempo excesivo de una sola muestra cause jitter del período, el sistema introduce una estrategia de compensación de período. Sea T_{sample} el período objetivo (en segundos), y $t_{elapsed}$ el tiempo real consumido en esta muestra, entonces el tiempo de dormir se calcula mediante la fórmula:

$$t_{sleep} = \max(0, T_{sample} - t_{elapsed}) \quad (16)$$

La fórmula 16 garantiza que el período real no sea inferior al valor establecido. Si una sola muestra ya ha excedido el tiempo límite, se omite el dormir y se pasa directamente a la siguiente ronda, priorizando la actualidad de los datos.

```
1 import time
2
3 SAMPLE_PERIOD = 2.0  # periodo de muestreo 2 segundos
4
5 def data_collection_loop(sensors, db, alarm_config):
6     """Ciclo principal de adquisicion de datos"""
7     while True:
8         t_start = time.time()
9
10        # Paso 1: leer parametros ambientales
11        env = read_environment_control()
12
13        # Paso 2: actualizar entorno de sensores
14        for sensor in sensors:
15            sensor.set_environment(env)
16
17        # Paso 3: ejecutar lectura de sensores
18        readings = {}
19        for sensor in sensors:
20            readings.update(sensor.read())
21
22        # Paso 4: formatear datos
23        record = format_sensor_data(readings)
24
25        # Paso 5: almacenar datos
26        db.insert_sensor_data(record)
27
28        # Paso 6: verificar alertas
29        check_alarms(record, alarm_config, db)
30
31        # Compensacion de periodo
32        t_elapsed = time.time() - t_start
33        t_sleep = max(0, SAMPLE_PERIOD - t_elapsed)
34        time.sleep(t_sleep)
```

Listing 6: Implementacion central del ciclo de adquisicion de datos

El Código 6 presenta la implementación completa de `data_collection_loop`. La constante `SAMPLE_PERIOD` especifica el período de muestreo, y la lógica de compensación de período suprime la deriva temporal. El diccionario `readings` transporta resultados intermedios entre los pasos, y el `record` formateado finalmente se almacena en la base de

datos y participa en la determinación de alertas.

5 Diseño e Implementación del Sistema de Software

La plataforma de hardware virtual STM32 y los controladores de sensores constituyen la base de operación inferior. Sobre esta base, los cuatro núcleos centrales — módulo de almacenamiento de datos, sistema de alertas, sistema de visualización Web y panel de control — trabajan de manera coordinada, asumiendo respectivamente las funciones de persistencia de datos, detección de anomalías, presentación de resultados y control remoto, formando conjuntamente la aplicación de software completa de la estación inteligente de monitoreo ambiental.

5.1 Módulo de Almacenamiento de Datos

El sistema de monitoreo debe primero escribir las lecturas de sensores adquiridas, el estado de operación y los eventos de alerta en medios de persistencia, para poder realizar retrospectiva histórica, análisis estadístico y auditoría de registros. SQLite, como el motor de base de datos de este sistema, no requiere un proceso de servicio independiente; toda la base de datos se almacena en un único archivo, siendo muy conveniente su despliegue y migración.

5.1.1 Diseño de la Estructura de Tablas de la Base de Datos

La base de datos contiene tres tablas principales: `sensor_data` (tabla de datos de sensores), `alarm_log` (tabla de registros de alertas) y `system_log` (tabla de registros del sistema).

(1) **Tabla `sensor_data`:** Utilizada para almacenar los datos de adquisición en tiempo real de cada sensor, es la tabla con mayor volumen de datos. Su estructura de diseño se muestra en la Tabla 7, conteniendo un total de 10 campos.

Cuadro 7: Estructura de la tabla sensor_data

Nombre del campo	Tipo de dato	Restricciones	Descripcion
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Clave primaria autoincremental
timestamp	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	Marca de tiempo de adquisicion
temperature	REAL		Valor de temperatura, unidad °C
humidity	REAL		Valor de humedad, unidad % RH
pressure	REAL		Valor de presion, unidad hPa
illuminance	REAL		Iluminancia, unidad lux
air_quality	REAL		Indice de calidad del aire, unidad ppm
noise_level	REAL		Nivel de ruido, unidad dB
cpu_temp	REAL		Temperatura interna del MCU, unidad °C
data_source	TEXT	DEFAULT 'simulation'	Identificador de fuente de datos

id adopta la estrategia de clave primaria autoincremental, asignando un identificador único a cada registro escrito; timestamp se rellena automáticamente con la hora actual por la base de datos en el momento de la inserción, proporcionando una base temporal para las búsquedas posteriores por período; los seis campos desde temperature hasta noise_level mapean uno a uno las seis cantidades físicas ambientales; cpu_temp almacena la lectura simulada del sensor de temperatura interna del MCU virtual; data_source está marcado por defecto como 'simulation', reservando una interfaz de expansión para la futura conexión de hardware real.

(2) **Tabla alarm_log**: Rastrea todo el ciclo de vida de las alertas desde su generación hasta su resolución. Los significados de los campos son los siguientes: id es la clave primaria; timestamp registra el momento del primer disparo de la alerta; sensor_type indica el sensor que generó la alerta; alarm_type distingue entre dos tipos de alertas: umbral alto (HIGH) y umbral bajo (LOW); threshold registra el límite establecido en el momento del disparo; actual_value guarda la lectura del sensor en el instante del disparo; status marca si la alerta se encuentra actualmente activa (ACTIVE) o ya resuelta (RESOLVED); resolved_at registra la marca de tiempo de la resolución de la alerta.

(3) **Tabla system_log**: Captura eventos clave y salidas de depuración durante la op-

eración del sistema. Los campos específicos son: id clave primaria; timestamp registra el momento de generación del registro; level clasifica la gravedad en cuatro niveles: DEBUG, INFO, WARNING, ERROR; module marca el módulo funcional que generó el registro; message contiene el texto específico; metadata almacena información estructurada adicional en formato JSON, facilitando el análisis automatizado posterior.

5.1.2 Interfaz sqlite3 de Python

La biblioteca estándar sqlite3 incluida en Python encapsula completamente la interfaz C de SQLite, permitiendo a los desarrolladores ejecutar SQL directamente, controlar los límites de transacciones y leer conjuntos de resultados, ahorrando los pasos de instalación de controladores de base de datos adicionales.

Cada acceso a la base de datos está envuelto por un gestor de contexto, y la conexión se libera automáticamente al finalizar la operación, eliminando el riesgo de fugas de manejadores. El `journal_mode` se establece explícitamente como WAL (Write-Ahead Logging), permitiendo que las operaciones de lectura y escritura avancen en paralelo; cuando el hilo de adquisición y el hilo de servicio Web acceden simultáneamente a la misma base de datos, el tiempo de espera de bloqueo se reduce significativamente.

5.1.3 Operaciones CRUD

El módulo de almacenamiento de datos expone al exterior un conjunto de interfaces CRUD que cubren las operaciones de creación, lectura, actualización y eliminación:

(1) **Create (Crear)**: `insert_sensor_data` recibe lecturas de sensores en forma de diccionario y las escribe en la tabla `sensor_data`. La consulta utiliza enlace parametrizado, de modo que el exterior no puede alterar la semántica SQL mediante la construcción de entradas maliciosas, bloqueando así el camino de los ataques de inyección.

(2) **Read (Leer)**: El módulo proporciona capacidades de búsqueda multidimensionales: `get_latest_reading()` devuelve el último punto de muestreo; `get_readings_by_range(start,end)` recupera todos los registros dentro de una ventana de tiempo dada; `get_readings_by_sensor(sensor_type,limit)` recupera los últimos N registros históricos según el tipo de sensor.

(3) **Update (Actualizar)**: `update_alarm_status` marca las alertas en estado ACTIVE como RESOLVED, y establece `resolved_at` como la hora actual del sistema, completando el ciclo cerrado de la alerta.

(4) **Delete (Eliminar)**: `delete_old_readings(before_date)` elimina registros vencidos según un límite de fecha, pudiendo ser activado automáticamente por tareas programadas o ejecutado manualmente por un administrador, evitando que los datos históricos continúen expandiéndose y causando presión de almacenamiento.

5.1.4 Consultas Estadísticas

Sobre las operaciones CRUD básicas, el módulo también incluye capacidades de análisis de agregación, permitiendo realizar análisis estadísticos de los datos de muestreo durante un período de tiempo. Las dimensiones de agregación comúnmente utilizadas son las siguientes:

(1) **Consulta de promedio (AVG)**: Calcula el promedio aritmético de las lecturas de sensores dentro de una ventana de tiempo seleccionada, caracterizando el nivel general de los parámetros ambientales durante ese período.

(2) **Consulta de valor mínimo (MIN)**: Extrae el valor de muestreo más bajo dentro de ese período, ayudando a descubrir valores anormalmente bajos o condiciones extremas.

(3) **Consulta de valor máximo (MAX)**: Extrae el valor de muestreo más alto dentro de ese período, ayudando a descubrir valores anormalmente altos o condiciones extremas.

(4) **Consulta de desviación estándar (STDEV)**: Mide el grado de dispersión de los puntos de muestreo alrededor de la media; cuanto mayor sea el valor, más intensa es la fluctuación ambiental.

La plantilla SQL para consultas estadísticas es la siguiente:

```
1 SELECT
2     AVG(temperature) as avg_temp,
3     MIN(temperature) as min_temp,
4     MAX(temperature) as max_temp,
5     AVG(humidity) as avg_humidity,
6     MIN(pressure) as min_pressure,
7     MAX(pressure) as max_pressure
8 FROM sensor_data
9 WHERE timestamp BETWEEN ? AND ?;
```

Listing 7: Plantilla SQL para consultas estadísticas

La plantilla SQL anterior utiliza funciones de agregación para resumir indicadores estadísticos de múltiples sensores de una sola vez. El límite temporal en la cláusula WHERE se pasa en forma de marcadores de posición, preservando tanto la flexibilidad de la consulta como evitando los riesgos de inyección derivados de la concatenación de cadenas.

5.2 Diseño del Sistema de Alertas

El sistema de alertas escanea continuamente las lecturas en tiempo real de cada sensor; una vez detectado un valor fuera de rango, genera una notificación para recordar al personal de operaciones que intervenga oportunamente para manejar la anomalía ambiental.

5.2.1 Configuración de Umbrales de Alerta

El diccionario ALARM_CONFIG mantiene límites superior e inferior independientes para cada sensor: `high_threshold` define el umbral alto, y `low_threshold` define el umbral bajo. Cuando la lectura excede el umbral alto, se genera una alerta de tipo HIGH; cuando cae por debajo del umbral bajo, se genera una alerta de tipo LOW; ambos tipos no se superponen.

El ejemplo de configuración de alerta por defecto es el siguiente:

```
1 ALARM_CONFIG = {
2     'temperature': {
3         'high_threshold': 35.0,    # umbral de alerta de alta
           temperatura 35C
4         'low_threshold': 10.0,    # umbral de alerta de baja
           temperatura 10C
5         'unit': 'C',
6         'name': 'Temperatura'
7     },
8     'humidity': {
9         'high_threshold': 80.0,    # umbral de alerta de alta humedad
           80%
10        'low_threshold': 20.0,    # umbral de alerta de baja humedad
           20%
11        'unit': '% RH',
12        'name': 'Humedad'
13    },
14    'pressure': {
15        'high_threshold': 1050.0, # umbral de alerta de alta presion
           1050 hPa
16        'low_threshold': 980.0,   # umbral de alerta de baja presion
           980 hPa
17        'unit': 'hPa',
18        'name': 'Presion'
19    },
20    'air_quality': {
21        'high_threshold': 500.0,   # umbral de alerta de calidad del
           aire 500 ppm
22        'low_threshold': None,     # sin umbral bajo
23        'unit': 'ppm',
24        'name': 'Calidad del aire'
25    },
26    # ... configuraciones de otros sensores
27 }
```

Listing 8: Ejemplo de configuracion de umbrales de alerta

Como se muestra en el Código 8, los elementos de configuración de cada sensor incluyen valores de umbral, unidades, nombres en español, etc. Cuando el `low_threshold` de un sensor se establece en `None`, significa que no se monitorea la alerta de valor bajo, solo la alerta de valor alto.

5.2.2 Lógica de Verificación de Alertas

La función `check_all` impulsa todo el flujo de determinación de alertas: lee secuencialmente la configuración de sensores en `ALARM_CONFIG`, compara el valor de muestreo actual con el intervalo de umbrales establecido para ese sensor, y activa o borra alertas según el resultado de la comparación. Los pasos específicos se descomponen de la siguiente manera:

(1) **Verificación iterativa:** Extrae secuencialmente los elementos de sensor de `ALARM_CONFIG`, y compara el valor de muestreo actual con el intervalo de umbrales establecido para ese sensor.

(2) **Determinación de umbral alto:** Cuando el valor de muestreo es mayor que `high_threshold`, el sistema busca si ya existe una alerta `HIGH` en estado `ACTIVE` para ese sensor: si existe, actualiza `actual_value`; si no existe, inserta un nuevo registro de alerta `ACTIVE`.

(3) **Determinación de umbral bajo:** Cuando el valor de muestreo es menor que `low_threshold`, se sigue un flujo simétrico al del umbral alto para activar una alerta de tipo `LOW`.

(4) **Recuperación de alerta:** Una vez que el valor de muestreo vuelve a caer dentro del intervalo de umbrales, el sistema marca en lote todas las alertas `ACTIVE` de ese sensor como `RESOLVED`, y escribe la marca de timestamp en `resolved_at`.

(5) **Persistencia:** Tanto la nueva alerta como la migración de estado se escriben inmediatamente en el disco a través de la interfaz de base de datos, garantizando que toda la cadena de alertas sea trazable.

```
1 def check_all(readings, config, db):
2     """
3     Verifica el estado de alerta de todos los sensores
4     Parametros:
5         readings: diccionario de lecturas actuales de sensores
6         config:    diccionario de configuracion de alertas
7         db:        instancia de operaciones de base de datos
8     """
9     for sensor, cfg in config.items():
10         value = readings.get(sensor)
```

```
11     if value is None:
12         continue
13
14     # Verificacion de umbral alto
15     high = cfg.get('high_threshold')
16     if high is not None and value > high:
17         trigger_alarm(db, sensor, 'HIGH', high, value)
18
19     # Verificacion de umbral bajo
20     low = cfg.get('low_threshold')
21     if low is not None and value < low:
22         trigger_alarm(db, sensor, 'LOW', low, value)
23
24     # Verificacion de recuperacion
25     if (high is None or value <= high) and \
26         (low is None or value >= low):
27         resolve_alarm(db, sensor)
```

Listing 9: Logica central de verificacion de alertas

Como se muestra en el Código 9, la función `check_all` implementa el flujo completo de verificación de alertas. `trigger_alarm` y `resolve_alarm` son funciones auxiliares, responsables de crear nuevas alertas y recuperar alertas existentes, respectivamente.

5.2.3 Persistencia de Registros de Alertas

Los registros de alertas preservan el contexto completo en el momento del disparo, incluyendo tipo de sensor, límite de umbral y valor de muestreo real, proporcionando evidencia original para la revisión y auditoría posterior. Su modelo de datos se describe en la Tabla 8.

Cuadro 8: Estructura de la tabla alarm_log

Nombre del campo	Tipo de dato	Restricciones	Descripcion
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Clave primaria autoincremental
timestamp	DATETIME	NOT NULL	Tiempo de disparo de la alerta
sensor_type	TEXT	NOT NULL	Tipo de sensor
alarm_type	TEXT	NOT NULL	Tipo de alerta: HIGH/LOW
threshold	REAL	NOT NULL	Umbral de disparo
actual_value	REAL	NOT NULL	Valor real en el momento del disparo
status	TEXT	DEFAULT 'ACTIVE'	Estado: ACTIVE/RESOLVED
resolved_at	DATETIME		Tiempo de resolucion

5.3 Sistema de Visualización Web

La capa de visualización Web construye el puente entre el usuario y la estación virtual de monitoreo: los datos de sensores subyacentes se renderizan en gráficos y paneles de instrumentos de manera intuitiva, mientras que los usuarios pueden ajustar parámetros ambientales y ver notificaciones de anomalías en el panel de alertas a través del panel de control.

5.3.1 Diseño de Rutas Flask

Flask asume las responsabilidades del servicio Web backend, exponiendo un conjunto de API RESTful para su consumo por el frontend. El método HTTP, la ruta y las responsabilidades de cada endpoint se resumen en la Tabla 9.

Cuadro 9: Diseño de rutas API RESTful de Flask

Metodo	Ruta	Funcion	Descripcion
GET	/	Pagina principal	Retorna la pagina de visualizacion Web
GET	/api/sensors/latest	Datos mas recientes	Retorna las lecturas mas recientes de todos los sensores
GET	/api/sensors/history	Datos historicos	Retorna datos historicos dentro de un rango de tiempo especificado
GET	/api/alarms/active	Alertas activas	Retorna la lista de alertas no resueltas actualmente
GET	/api/alarms/history	Historial de alertas	Retorna todos los registros de alertas (soporta paginacion)
POST	/api/control	Control ambiental	Recibe instrucciones de configuracion de parametros ambientales
GET	/api/stats	Estadisticas	Retorna informacion estadistica del periodo especificado

5.3.2 Formato de Respuesta JSON RESTful

Cada API retorna un cuerpo JSON que fijamente contiene tres claves de nivel superior: status, data, message. status toma el valor 'ok' o 'error', reflejando el resultado del procesamiento de la solicitud actual; data porta la carga útil real a nivel de negocio; message solo se rellena con una descripción de error legible en escenarios de excepción.

Ejemplo de respuesta exitosa:

```

1 {
2   "status": "ok",
3   "data": {
4     "temperature": 26.5,
5     "humidity": 58.2,
6     "pressure": 1013.8,
7     "illuminance": 320.0,
8     "air_quality": 120.5,
9     "noise_level": 45.3,
10    "timestamp": "2024-01-15T09:30:00"
11  },
12  "message": null
13 }
```

Listing 10: Ejemplo de respuesta exitosa de API

5.3.3 Configuración de Gráfico de Líneas Dual de Chart.js

El frontend selecciona Chart.js para renderizar curvas de tendencia en tiempo real. Para mostrar simultáneamente las tendencias de cambio de dos sensores relacionados en un único canvas, el sistema adopta un gráfico de líneas con doble eje Y (Dual Line Chart). A continuación se presenta el código de inicialización de Chart.js para la tendencia conjunta de temperatura y humedad:

```
1 const ctx = document.getElementById('tempHumChart').getContext('2d');
2 const tempHumChart = new Chart(ctx, {
3   type: 'line',
4   data: {
5     labels: [],      // arreglo de etiquetas de tiempo
6     datasets: [
7       {
8         label: 'Temperatura (C)',
9         data: [],
10        borderColor: 'rgb(255, 99, 132)',
11        backgroundColor: 'rgba(255, 99, 132, 0.1)',
12        yAxisID: 'y-temp',
13        tension: 0.3
14      },
15      {
16        label: 'Humedad (% RH)',
17        data: [],
18        borderColor: 'rgb(54, 162, 235)',
19        backgroundColor: 'rgba(54, 162, 235, 0.1)',
20        yAxisID: 'y-humidity',
21        tension: 0.3
22      }
23    ]
24  },
25  options: {
26    responsive: true,
27    interaction: { mode: 'index', intersect: false },
28    scales: {
29      'y-temp': {
30        type: 'linear',
31        display: true,
32        position: 'left',
33        title: { display: true, text: 'Temperatura (C)' }
34      },
35      'y-humidity': {
```

```
36         type: 'linear',
37         display: true,
38         position: 'right',
39         title: { display: true, text: 'Humedad (% RH)' }
40     }
41 }
42 }
43 });
```

Listing 11: Configuración de gráfico de líneas dual de Chart.js

El gráfico de líneas dual utiliza yAxisID para mapear la curva de temperatura al eje Y izquierdo y la curva de humedad al eje Y derecho, eliminando así el problema de compresión visual causado por las diferencias de magnitud. La tensión se establece en 0.3, preservando las características de los puntos de inflexión de los datos mientras hace que la línea poligonal sea más suave y natural en su conjunto.

5.3.4 Mecanismo de Sondeo de 1 Segundo

El frontend realiza sondeo activo al backend a través de setInterval con un período de 1000 milisegundos, obteniendo proactivamente las lecturas de sensores más recientes y el estado de alertas, logrando la actualización casi en tiempo real de la interfaz.

```
1 // Sondeo cada 1000 milisegundos (1 segundo)
2 setInterval(() => {
3     fetch('/api/sensors/latest')
4         .then(r => r.json())
5         .then(data => {
6             if (data.status === 'ok') {
7                 updateSensorCards(data.data);
8                 updateCharts(data.data);
9             }
10        })
11        .catch(err => console.error('Error de sondeo:', err));
12
13    fetch('/api/alarms/active')
14        .then(r => r.json())
15        .then(data => {
16            if (data.status === 'ok') {
17                updateAlarmPanel(data.data);
18            }
19        });
20 }, 1000);
```

Listing 12: Implementacion del mecanismo de sondeo del frontend

El frontend sondea en paralelo los dos endpoints `/api/sensors/latest` y `/api/alarms/active`, garantizando que los valores de las tarjetas, las curvas de tendencia y la lista de alertas siempre estén alineados con el estado del backend. En comparación con el push mediante WebSocket, el sondeo hace una ligera concesión en tiempo real, pero es de implementación ligera y amplia compatibilidad con navegadores, siendo completamente adecuado para escenarios de actualización a nivel de segundos. En iteraciones posteriores puede migrarse a SSE o WebSocket para reducir aún más la sobrecarga de conexiones.

5.4 Diseño del Panel de Control

Los usuarios modifican los parámetros ambientales virtuales directamente en el navegador a través del panel de control, observando en tiempo real los cambios en las salidas de los sensores bajo condiciones de operación específicas. Esta capacidad es particularmente crucial para demostraciones en clase y resolución de problemas.

5.4.1 Diseño del Mecanismo IPC

El ciclo de adquisición del MCU virtual reside en un hilo Python independiente, mientras que el servicio Web Flask se ejecuta en el hilo principal; ambos deben intercambiar instrucciones de control a través de algún canal entre hilos. El sistema selecciona un esquema de IPC basado en archivos compartidos: `env_control.txt` actúa como el medio de intercambio de datos bidireccional.

El IPC basado en archivos fue adoptado basándose en cuatro consideraciones: primero, no requiere la introducción de dependencias adicionales como Redis o ZeroMQ, reduciendo la complejidad del sistema; segundo, el contenido del archivo se preserva incluso después del reinicio del proceso, poseyendo naturalmente la capacidad de persistencia de estado; tercero, el formato de texto facilita que los desarrolladores lo visualicen y modifiquen manualmente, con bajo costo de depuración; cuarto, la lectura y escritura de archivos es una capacidad básica de la gran mayoría de los lenguajes de programación, minimizando la resistencia a la migración al introducir módulos heterogéneos en el futuro.

El archivo `env_control.txt` adopta un formato simple de pares clave-valor, con un parámetro por línea, en el formato `key=value`:

```
1 temperature=25.0
2 humidity=60.0
3 pressure=1013.25
4 illuminance=500.0
5 air_quality=100.0
```



```
6 noise_level=40.0
7 wind_speed=5.0
```

Listing 13: Ejemplo de formato del archivo de control ambiental

env_control.txt mantiene 7 parámetros ambientales. El hilo de adquisición lee este archivo al inicio de cada ciclo de muestreo, actualizando la entrada ambiental de los sensores virtuales a los valores más recientes; el backend Web reescribe los pares clave-valor correspondientes tras recibir una solicitud POST, y los nuevos parámetros entran en vigor en el siguiente ciclo de adquisición.

5.4.2 Análisis del Comando set

El endpoint /api/control recibe cuerpos de solicitud POST en formato JSON, extrae los parámetros ambientales a modificar y los reescribe en env_control.txt. La lógica de análisis sigue la semántica de actualización parcial: se actualizan tantas claves como aparezcan en el cuerpo de la solicitud, y las demás claves mantienen su valor original, evitando que el usuario deba enviar parámetros completos cada vez.

```
1 @app.route('/api/control', methods=['POST'])
2 def set_environment():
3     """Establece los parametros ambientales"""
4     data = request.get_json()
5     if not data:
6         return jsonify({'status': 'error', 'message': 'Sin datos'}),
           400
7
8     # Lee los parametros ambientales actuales
9     env = read_env_file()
10
11     # Actualiza los parametros especificados
12     valid_keys = ['temperature', 'humidity', 'pressure',
13                  'illuminance', 'air_quality', 'noise_level',
14                  'wind_speed']
15     for key in valid_keys:
16         if key in data:
17             env[key] = float(data[key])
18
19     # Escribe en el archivo
20     write_env_file(env)
21
22     return jsonify({'status': 'ok', 'data': env})
```

Listing 14: Interfaz de configuracion de parametros ambientales

La ruta de ejecución de `set_environment` se divide en tres pasos: primero carga el archivo de entorno existente, luego sobrescribe las entradas correspondientes según los pares clave-valor de la solicitud, y finalmente escribe el resultado de vuelta al disco. La lista blanca `valid_keys` combinada con la conversión forzada a float bloquea claves ilegales y entradas de tipos erróneos.

5.4.3 Restricciones del Rango de Parámetros

La simulación de sensores carece de capacidad de tolerancia a fallas física para entradas extremas, por lo que el backend delimita rangos válidos para cada uno de los 7 parámetros ambientales. Cuando el usuario envía valores fuera de rango, la API retorna un código de estado 400 con un mensaje de error, solicitando la corrección y reintento. Los límites superior e inferior y los valores por defecto de cada parámetro se muestran en la Tabla 10.

Cuadro 10: Rangos validos de parametros ambientales

Nombre del parametro	Minimo	Maximo	Por defecto	Unidad
temperature	-40.0	+85.0	25.0	°C
humidity	0.0	100.0	50.0	% RH
pressure	300.0	1100.0	1013.25	hPa
illuminance	0.0	65535.0	500.0	lux
air_quality	0.0	10000.0	100.0	ppm
noise_level	0.0	150.0	40.0	dB
wind_speed	0.0	100.0	5.0	m/s

El diseño de los intervalos de cada parámetro se alinea estrictamente con las hojas de datos de sensores reales y las leyes físicas: los límites superior e inferior de temperatura coinciden con el rango de operación nominal del BMP280; la humedad está limitada a 0–100%, ya que valores fuera de este intervalo carecen de significado físico; el intervalo de presión abarca desde la baja presión de alta montaña hasta la alta presión a nivel del mar; el límite superior de iluminancia de 65535 lux coincide exactamente con el rango completo del BH1750 en modo H-Res. Estos límites interceptan la gran mayoría de las entradas erróneas, manteniendo la razonabilidad de las salidas simuladas.

6 Pruebas y Verificación del Sistema

Para verificar si los módulos de la plataforma de hardware virtual STM32, controladores de sensores, almacenamiento de datos, sistema de alertas y visualización Web alcanzan los indicadores esperados, es necesario construir un sistema de pruebas sistemático y ejecutar verificaciones multidimensionales. A continuación se explica desde cinco perspectivas: entorno de pruebas, cobertura funcional, referencias de rendimiento, precisión de sensores y confiabilidad de alertas.

6.1 Entorno de Pruebas

La configuración del entorno de pruebas de este sistema se muestra en la Tabla 11.

Cuadro 11: Configuración del entorno de pruebas del sistema

Elemento	Configuración
Sistema operativo	Windows 11 Professional 64-bit
Procesador	Intel Core i7-12700H @ 2.3 GHz
Memoria	16 GB DDR5
Version de Python	Python 3.11.6
Version de Flask	Flask 3.0.0
Version de SQLite	SQLite 3.42.0
Navegador	Google Chrome 120.0

La máquina de pruebas es una computadora portátil equipada con Intel Core i7-12700H y 16 GB de memoria DDR5, una configuración básicamente equivalente a los entornos de desarrollo actuales. Todo el flujo de pruebas se ejecuta localmente en circuito cerrado, excluyendo las interferencias de la latencia de la red de área amplia y las fluctuaciones de carga de servidores remotos.

6.2 Pruebas Funcionales

Las pruebas funcionales verifican el comportamiento del sistema elemento por elemento según las especificaciones de requisitos. El diseño de casos de uso cubre seis módulos: inicialización del MCU virtual, lectura de datos de sensores, carga de páginas Web, extracción de datos de API, activación de umbrales de alertas y envío de parámetros del panel de control. Los resultados de las pruebas se resumen en la Tabla 12.

Cuadro 12: Resultados de las pruebas funcionales

ID	Elemento de prueba	Pasos de prueba	Resultado
FT-01	Inicio del MCU	Inicializar instancia de VirtualSTM32	El sistema inicia
FT-02	Habilitacion del ADC	Configurar reloj del ADC y bit de habilitacion	El ADC entra
FT-03	Lectura del DHT11	Invocar metodo dht11.read()	Retorna valores de tem
FT-04	Lectura del MQ-135	Esperar precalentamiento e invocar read()	Retorna valor de co
FT-05	Lectura del BH1750	Enviar comando de medicion y leer	Retorna valor de
FT-06	Lectura del BMP280	Leer ID del chip y obtener datos	Retorna presio
FT-07	Lectura del ruido	Invocar metodo noise.read()	Retorna v
FT-08	Acceso a pagina Web principal	Navegador accede a http://localhost:5000/	La pagina car
FT-09	Obtencion de datos de API	Solicitud GET a /api/sensors/latest	Retorna c
FT-10	Activacion de alerta de alta temperatura	Establecer temperature=40	Se genera reg
FT-11	Activacion de alerta de baja presion	Establecer pressure=920	Se genera reg
FT-12	Operacion del panel de control	Modificar parametros ambientales desde la interfaz Web	El archivo de para

Los 12 casos de uso de la Tabla 12 pasaron todos la aceptación. La capa de controladores de sensores (FT-03 a FT-07) confirma que los cinco sensores virtuales pueden retornar correctamente las cantidades físicas; la capa de servicio Web (FT-08, FT-09) confirma que el renderizado de páginas y las interfaces JSON responden normalmente; la capa de lógica de alertas (FT-10, FT-11) confirma que tanto los excesos de umbral alto como de umbral bajo pueden ser capturados; la capa de control (FT-12) confirma que el archivo de parámetros ambientales puede ser reescrito remotamente.

6.3 Pruebas de Rendimiento

Las pruebas de rendimiento se enfocan en la latencia de respuesta y el uso de recursos, verificando si el sistema aún puede satisfacer los requisitos de tiempo real de monitoreo a nivel de segundos bajo operación continua.

6.3.1 Tiempo de Respuesta de la API

Utilizando la biblioteca requests de Python, se enviaron 100 solicitudes consecutivas a cada endpoint del backend Flask, registrando el tiempo de ida y vuelta de cada solicitud y calculando los valores estadísticos. Los resultados de medición se muestran en la Tabla 13.

Cuadro 13: Resultados de pruebas de tiempo de respuesta de la API

Endpoint de API	Tiempo promedio	Minimo	Maximo
GET /api/sensors/latest	12.5 ms	8.2 ms	28.4 ms
GET /api/sensors/history	45.3 ms	32.1 ms	112.6 ms
GET /api/alarms/active	9.8 ms	6.5 ms	22.1 ms
POST /api/control	15.2 ms	10.3 ms	35.7 ms
GET /api/stats	38.6 ms	25.4 ms	98.3 ms

El tiempo de respuesta promedio de todos los endpoints se controla dentro de 100 milisegundos, cumpliendo con la línea base de rendimiento preestablecida. /api/sensors/latest y /api/alarms/active solo consultan el último registro individual, con un tiempo promedio de aproximadamente 10 milisegundos; /api/sensors/history y /api/stats necesitan escanear intervalos de tiempo y ejecutar operaciones de agregación, con el tiempo promedio aumentando a 40–45 milisegundos, donde la sobrecarga adicional proviene principalmente de la búsqueda por intervalo y el cálculo de agrupación en la base de datos.

6.3.2 Estabilidad del Período de Adquisición

El jitter del período de adquisición se reflejará directamente en la suavidad de las curvas de tendencia, y también afectará la precisión temporal de la determinación de alertas. El experimento registró continuamente el tiempo real de ejecución de 100 ciclos, tomando el período objetivo $T_{sample} = 2.0$ segundos como referencia, evaluando la magnitud de la deriva.

El período promedio de 100 muestreos fue de 2.003 segundos, con una desviación estándar de 0.018 segundos y una deriva máxima positiva de +0.052 segundos. La causa raíz del jitter reside en la fluctuación del tiempo consumido por la escritura SQLite y la lógica de determinación de alertas en una sola muestra; incluso cuando los datos históricos se acumulan a cierta escala, el modo WAL aún puede mantener el tiempo de espera de bloqueo en un nivel relativamente bajo. Desde una perspectiva general, la estabilidad del período es suficiente para soportar el monitoreo en tiempo real, y las curvas de tendencia pueden reproducir fielmente las tendencias de los parámetros ambientales.

6.3.3 Uso de Memoria

Utilizando tracemalloc incorporado en Python y la biblioteca de terceros psutil, se realizó un seguimiento continuo de la huella de memoria del sistema. Después de una operación

continua de 1 hora, los detalles de uso de cada módulo se muestran en la Tabla 14.

Cuadro 14: Resultados de pruebas de uso de memoria

Modulo	Uso de memoria
Servicio Web Flask	Aproximadamente 45 MB
Instancias de MCU virtual y sensores	Aproximadamente 12 MB
Hilo de adquisicion de datos	Aproximadamente 8 MB
Cache de base de datos SQLite	Aproximadamente 5 MB
Recursos de pagina frontend	Aproximadamente 3 MB (lado del navegador)
Total	Aproximadamente 73 MB

El consumo total de memoria del sistema es de aproximadamente 73 MB, de los cuales el servicio Web Flask ocupa aproximadamente 45 MB, una parte generada principalmente por el tiempo de ejecución del intérprete de Python y la estructura propia del framework Flask. Los 73 MB representan una carga extremadamente ligera en entornos de computación modernos, pudiendo desplegarse sin problemas incluso en dispositivos periféricos como Raspberry Pi.

6.4 Verificación de Precisión de Sensores

La credibilidad de la estación virtual de monitoreo depende en gran medida de la precisión de la simulación de los sensores. Para cuantificar este indicador, se realiza una comparación transversal del desempeño del error de cada sensor virtual con la precisión nominal especificada en las hojas de datos oficiales.

6.4.1 Análisis de Comparación de Precisión

Las diferencias de precisión entre los sensores virtuales y los chips reales se resumen en la Tabla 15.

Cuadro 15: Comparacion de precision entre sensores virtuales y sensores reales

Sensor	Parametro medido	Precision real	Precision simulada	Desviacion
DHT11	Temperatura	$\pm 2^{\circ}\text{C}$	$\pm 0.8^{\circ}\text{C}$	Mejor que real
DHT11	Humedad	$\pm 5\% \text{ RH}$	$\pm 1.5\% \text{ RH}$	Mejor que real
MQ-135	Concentracion de gas	$\pm 15\%$	$\pm 8\%$	Mejor que real
BH1750	Iluminancia	$\pm 20\%$	$\pm 5\%$	Mejor que real
BMP280	Presion	$\pm 1 \text{ hPa}$	$\pm 0.5 \text{ hPa}$	Mejor que real
BMP280	Temperatura	$\pm 0.5^{\circ}\text{C}$	$\pm 0.3^{\circ}\text{C}$	Mejor que real
Sensor de ruido	Nivel de presion sonora	$\pm 3 \text{ dB}$	$\pm 1.5 \text{ dB}$	Mejor que real

De la Tabla 15 se puede observar que los intervalos de error de todos los sensores virtuales son más estrechos que la precisión nominal de los chips reales. La simulación por software excluye fuentes de interferencia física como deriva térmica, envejecimiento de dispositivos, acoplamiento electromagnético y riple de fuente de alimentación; el ruido de salida solo es inyectado por el generador de números aleatorios gaussianos, por lo que la consistencia y repetibilidad son significativamente superiores al hardware real.

Que la precisión de simulación sea mayor que la precisión real no disminuye el valor de la plataforma para la enseñanza y la verificación. En entornos de clase, el menor ruido permite a los estudiantes identificar más fácilmente las leyes de correspondencia entre la respuesta del sensor y las variables ambientales, evitando que los errores aleatorios del hardware real oculten las características clave. Si se desea acercarse a las condiciones reales de trabajo, solo es necesario aumentar la desviación estándar σ del ruido gaussiano para devolver los sensores virtuales a un nivel de error similar al de los chips reales, realizando pruebas con un carácter más práctico.

6.4.2 Pruebas de Estabilidad a Largo Plazo

Se seleccionó el BMP280 virtual para realizar una prueba de estabilidad continua de 24 horas: los parámetros ambientales se fijaron en $P = 1013.25$ hPa, $T = 25.0^\circ\text{C}$, con un período de muestreo de 2 segundos. Los resultados estadísticos de 43200 puntos de datos muestran que la presión promedio fue de 1013.28 hPa, con una desviación estándar de 0.42 hPa, una desviación extrema positiva máxima de +1.8 hPa y una desviación extrema negativa máxima de -1.5 hPa. La distribución de los puntos de muestreo sigue una ley normal, sin aparecer deriva monótona ni desviación sistemática, demostrando la estabilidad numérica del sensor virtual durante la operación a largo plazo.

6.5 Verificación de la Función de Alertas

La confiabilidad del módulo de alertas determina si el usuario puede obtener notificaciones y tomar acciones de manera oportuna cuando el entorno se deteriora. A continuación se diseñan dos escenarios típicos para verificar el flujo completo de activación, presentación y recuperación de alertas.

6.5.1 Escenario de Alerta de Alta Temperatura

Configuración del escenario: a través del panel de control se eleva la temperatura ambiental a 40°C , valor que ha excedido el umbral alto de temperatura de 35°C .

Pasos de la prueba:

1. A través del panel de control, establecer el parámetro temperature en 40.0.
2. Esperar a que el ciclo de adquisición de datos ejecute al menos un período (aproximadamente 2 segundos).

3. Consultar la tabla `alarm_log`, verificando si se ha generado un nuevo registro de alerta de temperatura.
4. Acceder al panel de alertas de la interfaz Web, confirmando que la información de alerta se muestra correctamente.
5. Restaurar `temperature` a 25.0, y esperar la recuperación de la alerta.

Registro de observaciones: tras establecer `temperature` en 40.0, el sistema identificó exitosamente el exceso de umbral en el segundo ciclo de muestreo, agregando un nuevo elemento con `status='ACTIVE'` y `alarm_type='HIGH'` a `alarm_log`, con `actual_value` registrado como 40.1°C (ya superpuesto con ruido de simulación), y `threshold` de 35.0°C. El panel de alertas de la Web presentó esta anomalía destacada con una tarjeta roja. Tras restaurar la temperatura al rango normal, aproximadamente 2 segundos después, el mismo registro migró automáticamente al estado `RESOLVED`, y el campo `resolved_at` registró el momento de la desactivación.

6.5.2 Escenario de Alerta de Baja Presión

Configuración del escenario: a través del panel de control se reduce la presión ambiental a 920 hPa, valor que ha caído por debajo del umbral bajo de presión de 980 hPa.

Pasos de la prueba:

1. A través del panel de control, establecer el parámetro `pressure` en 920.0.
2. Esperar a que el ciclo de adquisición de datos ejecute al menos un período.
3. Consultar la tabla `alarm_log`, verificando si se ha generado un nuevo registro de alerta de presión.
4. Acceder al panel de alertas de la interfaz Web, confirmando que la información de alerta se muestra correctamente.
5. Restaurar `pressure` a 1013.25, y esperar la recuperación de la alerta.

Registro de observaciones: tras establecer `pressure` en 920.0, el sistema identificó correctamente el exceso del umbral bajo, generando un elemento de alerta con `alarm_type='LOW'`, con `actual_value` de 919.6 hPa. El frontend distinguió la alerta baja con una etiqueta naranja (las alertas altas utilizan rojo). Tras la recuperación de la presión, la alerta fue marcada automáticamente como `RESOLVED` en ciclos de muestreo posteriores.

6.5.3 Escenario de Alertas Concurrentes Múltiples

Verificación del escenario concurrente: simultáneamente se eleva `temperature` a 42.0°C y `air_quality` a 800 ppm, forzando la generación de alertas `HIGH` tanto en la dimensión de temperatura como en la de calidad del aire. El sistema generó registros `ACTIVE`

independientes para ambos sensores, sin sobrescritura a nivel de base de datos; el panel de alertas de la Web mostró ambas anomalías en paralelo, con colores correspondientes a cada sensor; durante la recuperación posterior de cada una, la migración de estados también fue mutuamente independiente. Este escenario demuestra que el módulo de alertas mantiene la corrección lógica y la estabilidad operativa bajo condiciones de concurrencia multi-fuente.

6.6 Visualización de la Interfaz Web

La arquitectura de información y el diseño visual de la interfaz Web determinan directamente la eficiencia con la que los usuarios obtienen información del estado ambiental. El frontend de este sistema se divide en seis áreas funcionales:

(1) **Tarjetas de datos de sensores:** En la primera pantalla se disponen horizontalmente 6 tarjetas de datos, cada una con el nombre del sensor en la parte superior, el valor actual y la unidad ampliados en el centro, y una flecha de tendencia en la parte inferior ($\uparrow$ ascendente, $\downarrow$ descendente, $\rightarrow$ estable). Cuando la lectura se encuentra dentro del rango seguro, el fondo de la tarjeta es verde; cuando se acerca al umbral, cambia gradualmente a amarillo, formando una advertencia visual intuitiva.

(2) **Gráfico de tendencia de temperatura y humedad:** En la zona central izquierda se coloca un gráfico de líneas con doble eje Y, donde la serie roja rastrea la tendencia de temperatura y la serie azul rastrea la tendencia de humedad, con el eje horizontal representando el tiempo de muestreo. Al pasar el mouse sobre un punto de datos, aparece un Tooltip mostrando el valor preciso, y la actualización de la curva va acompañada de una animación de transición suave.

(3) **Gráfico de calidad del aire e iluminancia:** En la zona central derecha se coloca simétricamente otro grupo de gráficos de líneas con doble eje Y, donde la curva púrpura mapea la calidad del aire y la curva amarilla mapea la iluminancia. Dado que la diferencia de magnitud entre ppm y lux es significativa, escalas independientes a izquierda y derecha evitan que la secuencia de menor magnitud sea comprimida hasta parecer una línea recta.

(4) **Panel de información estadística:** En la parte inferior izquierda se resumen indicadores agregados en tres granularidades de tiempo: 1 hora, 24 horas y 7 días, cubriendo dimensiones como promedios y valores extremos. Los datos se obtienen dinámicamente a través de `/api/stats`, y el usuario puede cambiar la ventana estadística mediante un menú desplegable.

(5) **Panel de alertas:** En la parte inferior derecha se divide en dos secciones: la sección superior enumera las alertas en estado ACTIVE en tiempo real, con fondo rojo/naranja para llamar la atención, y cada alerta indica el tipo, sensor, momento de disparo y valor fuera de rango; la sección inferior muestra los registros históricos en orden cronológico inverso, soportando navegación por páginas.

(6) **Panel de control:** En la barra lateral derecha se integran 7 grupos de controles de ajuste, correspondientes respectivamente a temperatura, humedad, presión, iluminancia, calidad del aire, ruido y velocidad del viento. Cada control está acompañado de la lectura actual y el intervalo válido, y las entradas fuera de rango son interceptadas inmediatamente con un mensaje de error amigable.

El frontend adopta un diseño de cuadrícula responsivo, reorganizando automáticamente las tarjetas y gráficos según el ancho del viewport, manteniendo la legibilidad tanto en escritorio como en dispositivos móviles. La paleta de colores utiliza gris claro como base, azul-verde como tono principal, y rojo-naranja para alertas, con niveles progresivos que ayudan a los usuarios a localizar la información clave en 3 segundos.

7 Resumen y Perspectivas

7.1 Resumen del Trabajo de la Tesis

Esta investigación toma la virtualización de la estación inteligente de monitoreo ambiental STM32 como objeto central, completando un trabajo integral desde la teoría hasta el prototipo. Los productos principales pueden resumirse en los siguientes tres puntos:

(1) **Construcción de una plataforma de hardware virtual STM32 de alta fidelidad.** Tomando el STM32F103C8T6 como referencia, se implementó en Python un conjunto completo de plataforma virtual MCU que abarca la simulación del núcleo Cortex-M3, el sistema de mapeo de memoria de tres segmentos (Flash/SRAM/Registros), el temporizador SysTick, GPIO, ADC, I2C, UART y TIM. Esta plataforma no depende de ningún hardware físico, y puede ejecutar programas embebidos directamente en Windows, Linux y macOS. Los experimentos de verificación demuestran que el comportamiento de lectura y escritura de los registros virtuales coincide altamente con la descripción del manual de referencia del STM32 real, siendo suficiente para soportar la enseñanza en clase y la verificación de prototipos tempranos.

(2) **Establecimiento de un marco unificado de virtualización de sensores ambientales.** Sobre el hardware virtual se completó la portación de controladores para cinco tipos de sensores ambientales comunes: DHT11, MQ-135, BH1750, BMP280 y sensor de ruido. A nivel de marco se unificaron el mapeo de parámetros ambientales, la conversión de cantidades físicas y el mecanismo de inyección de ruido gaussiano $Y = X + N(0, \sigma^2)$, de modo que la salida simulada sigue las leyes físicas y retiene características aleatorias reales. Cada controlador reproduce completamente los protocolos de comunicación de bus único e I2C, la secuencia de encendido y la cadena de acondicionamiento de señales, con un error integral superior a la precisión nominal especificada en las hojas de datos oficiales.

(3) **Implementación de un sistema completo de visualización y control de**

datos Web. El backend basado en Flask proporciona API RESTful, SQLite se encarga del almacenamiento persistente de datos de sensores, registros de alertas y registros del sistema; el frontend utiliza HTML/CSS/JavaScript nativo junto con Chart.js para completar gráficos de líneas en tiempo real, retrospectiva histórica, ventanas emergentes de alertas y controles deslizantes de parámetros ambientales. El panel de control y el MCU virtual logran la comunicación de circuito cerrado a través del archivo compartido `env_control.txt`; el usuario puede arrastrar controles en el frontend y observar los cambios en las lecturas de los sensores en el siguiente ciclo de muestreo.

La etapa de pruebas confirmó que: la plataforma virtual puede operar de manera estable 24/7, la deriva del período de adquisición se controla dentro de ± 0.052 segundos, el tiempo de respuesta promedio de la API Web es inferior a 100 milisegundos, y el consumo total de memoria del sistema es de solo aproximadamente 73 MB. El módulo de alertas puede capturar y archivar con precisión tanto los excesos de umbral alto como de umbral bajo, y la jerarquía de información del frontend es clara. Esta solución de software puro reduce los costos de acceso a los experimentos embebidos al mínimo, y al mismo tiempo proporciona un entorno de verificación de bajo costo y cero consumo de hardware para la iteración rápida de proyectos IoT en etapas tempranas.

7.2 Perspectivas de Trabajo Futuro

La versión actual de la estación virtual de monitoreo ambiental ya cubre los escenarios centrales de enseñanza y verificación de prototipos, pero aún existen cinco caminos que merecen una exploración más profunda en términos de profundidad de ingeniería y conexión con el ecosistema:

(1) **Simulación del sistema operativo de tiempo real FreeRTOS.** La plataforma actual solo soporta programas bare-metal, y la planificación de tareas depende completamente de sondeo e interrupciones. El siguiente paso podría ser portar el planificador de FreeRTOS para soportar la prioridad de apropiación, round-robin por quantum de tiempo, sincronización por semáforos, comunicación por colas de mensajes y asignación dinámica de memoria, permitiendo a los estudiantes experimentar la programación concurrente multi-tarea en un entorno virtual, acortando la distancia cognitiva del aula al campo industrial.

(2) **Soporte del protocolo MQTT.** El sondeo HTTP es aceptable en escenarios a nivel de segundos, pero la sobrecarga de conexión y el retraso de push limitan su escalabilidad en escenarios IoT de alta concurrencia. Tras introducir MQTT, el MCU virtual actúa como publicador enviando datos de sensores al Broker, y el frontend Web y la aplicación móvil actúan como suscriptores recibiendo actualizaciones de temas, logrando un verdadero push bajo demanda. MQTT, como el protocolo estándar de facto en el campo del IoT, su incorporación mejorará directamente la capacidad de conexión de la

plataforma con sistemas industriales reales.

(3) **Expansión a más tipos de sensores.** La biblioteca actual de sensores cubre cinco dimensiones: temperatura/humedad, presión, iluminancia, calidad del aire y ruido. En el futuro se pueden continuar integrando módulos como el detector de dióxido de carbono MH-Z19C, el módulo de partículas PMS5003, el módulo ultravioleta GUVAS12SD, sensores de humedad del suelo y pluviómetros de balancín. Con la ampliación de variedades, la plataforma puede integrarse sin problemas en más campos verticales como invernaderos de agricultura inteligente, monitoreo ambiental reticularizado urbano y seguridad e higiene ocupacional en fábricas.

(4) **Simulación de modos de bajo consumo.** El STM32F103C8T6 incorpora tres niveles de estado de bajo consumo: Sleep, Stop y Standby; el rendimiento de autonomía en escenarios alimentados por batería depende en gran medida de la gestión precisa de estos estados. En el futuro, la plataforma virtual podría simular el flujo completo de control de reloj, desconexión de periféricos y mapeo de fuentes de despertar, permitiendo a los estudiantes comprender el equilibrio entre consumo de energía y latencia de respuesta sin agotar baterías reales.

(5) **Desarrollo de aplicación móvil.** Aunque la interfaz Web actual ya posee capacidad multiplataforma, aún está limitada por el sandbox del navegador en aspectos como pantallas táctiles pequeñas, persistencia en segundo plano y notificaciones locales. Tras desarrollar una aplicación complementaria basada en Flutter o React Native, se podría acceder a notificaciones a nivel de sistema, reproducción de voz de alertas anormales, y vincular datos de monitoreo con ubicación GPS, logrando una verdadera percepción ambiental portátil.

Desde una perspectiva de enseñanza, los medios de virtualización eliminan las restricciones rígidas de la adquisición de hardware y la programación de laboratorios, permitiendo a los estudiantes reproducir todo el flujo de desarrollo embebido en sus computadoras personales en cualquier momento; desde una perspectiva de ingeniería, el entorno de verificación de software puro permite iterar rápidamente esquemas de arquitectura a un costo extremadamente bajo en etapas tempranas del proyecto. A medida que la industria del IoT mantiene una demanda sostenida de talento en sistemas embebidos, este tipo de plataformas virtuales de alta fidelidad desempeñará un papel cada vez más crucial en acortar los ciclos de formación de talentos y reducir los costos de prueba y error de la innovación.

Referencias

- [1] Liu Huoliang, Yang Sen. STM32 Library Development Practical Guide: Based on STM32F4[M]. Beijing: China Machine Press, 2019.
- [2] Zhengdian Yuanzi. STM32F1 Development Guide: Library Function Version[M]. Guangzhou: Zhengdian Yuanzi, 2022.
- [3] Wang Lihua, Zhang Minghui. Design and Implementation of Embedded System Virtual Experiment Platform[J]. Computer Education, 2021(5): 112-117.
- [4] Chen Zhigang, Li Xiaofeng. Review of Research on Intelligent Environmental Monitoring System Based on Internet of Things[J]. Transducer and Microsystem Technologies, 2020, 39(8): 1-4.
- [5] Zhou Runjing, Zhang Lina. STM32 Simulation and Application Examples Based on Proteus[M]. Beijing: Publishing House of Electronics Industry, 2018.
- [6] ARM Limited. Cortex-M3 Technical Reference Manual[EB/OL]. ARM Developer, 2021.
- [7] STMicroelectronics. RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs[EB/OL]. ST Community, 2021.
- [8] Sensirion AG. SHT30-DIS Datasheet: Humidity and Temperature Sensor[EB/OL]. Sensirion, 2020.
- [9] Bosch Sensortec. BMP280 Digital Pressure Sensor Datasheet[EB/OL]. Bosch, 2020.
- [10] ROHM Semiconductor. BH1750FVI-E Datasheet: Digital 16bit Serial Output Type Ambient Light Sensor[EB/OL]. ROHM, 2019.
- [11] Hanwei Electronics. MQ-135 Gas Sensor Datasheet[EB/OL]. Hanwei, 2018.
- [12] Flask Documentation. Flask 3.0 Quick Start[EB/OL]. Pallets Projects, 2023. <https://flask.palletsprojects.com/>
- [13] Mozilla Developer Network. Fetch API Web APIs[EB/OL]. MDN Web Docs, 2023. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [14] Chart.js Documentation. Chart.js 4.x Getting Started[EB/OL]. Chart.js, 2023. <https://www.chartjs.org/docs/>
- [15] Wang Zhihua, Liu Yang. Design and Implementation of IoT Environmental Monitoring System Based on MQTT Protocol[J]. Internet of Things Technologies, 2022, 12(3): 45-48.

- [16] Li Jianguo, Zhang Wei. Research on Application of Simulation Technology in Embedded System Teaching[J]. Journal of Electrical and Electronic Education, 2021, 43(2): 78-82.
- [17] SQLite Consortium. SQLite Documentation[EB/OL]. SQLite, 2023. <https://www.sqlite.org/docs.html>
- [18] Zhao Minghua, Sun Li. Application of Virtual Instrument Technology in Sensor Teaching[J]. Experimental Technology and Management, 2020, 37(6): 156-160.
- [19] QEMU Project. QEMU Documentation[EB/OL]. QEMU, 2023. <https://www.qemu.org/documentation/>
- [20] Yang Liqiang, Wang Haitao. Design of Web-based Remote Embedded Experiment Teaching Platform[J]. Computer Engineering and Applications, 2019, 55(14): 245-250.